

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



REPORT
CAPSTONE PROJECT

BIASED-GUIDANCE SAMPLING BASED
PATH PLANNING FOR AUTONOMOUS
MOBILE ROBOTS

MAJOR: COMPUTER SCIENCE

SUPERVISOR(s) : LE HONG TRANG
TRUONG QUYNH CHI

———— o0o ————

STUDENT : NGUYEN XUAN LOC – 2252461
CAO QUE PHUONG – 2252652

HO CHI MINH CITY, March 2026

Acknowledgement

We would like to express our sincere gratitude to Assoc. Prof. Le Hong Trang, M.Sc Truong Quynh Chi and M.Sc Kieu Van Xuan for their invaluable guidance, insightful feedback, and continuous support throughout the development of this project. Their expertise in path planning and robot operation, along with their careful supervision, played a crucial role in shaping the technical direction, research methodology, and overall quality of this work. We are also grateful for the academic environment provided by the Faculty of Computer Science and Engineering, Ho Chi Minh City University of Technology - Vietnam National University Ho Chi Minh City, which enabled us to conduct this research. Finally, we would like to extend our heartfelt appreciation to our families for their unwavering encouragement, patience, and support throughout our academic journey. Their understanding and motivation were essential to the successful completion of this project.

Abstract

Path planning is a fundamental problem in the field of robotics, aiming to find a path between a start point and a destination within the robot's workspace while avoiding obstacles and minimizing cost. An effective approach that has been widely studied and practically applied to this problem is sampling-based methods, with classic algorithms including Probabilistic Roadmaps (PRM) [4] and Rapidly-Exploring Random Trees (RRT). Numerous improved variants of these algorithms have been proposed.

This research focuses on improving the sampling process to enhance path planning performance, especially in complex environments such as those with many obstacles or narrow passages. The proposed approach is to perform sampling using guided and directed strategies to reduce the size of the tree during the search process. This helps to quickly find a path with better path length.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Goals	2
1.3	Scope	3
1.4	Report Structure	5
2	Theoretical Background	6
2.1	Overview of Path Planning Algorithms	6
2.1.1	Graph-based Approaches	6
2.1.2	Sampling-based Approaches	7
2.2	Configuration Space and Path Planning	8
2.3	Rapidly-exploring Random Trees (RRT)	9
2.4	Fundamentals of Bidirectional RRT	9
2.5	RRT* and Path Optimization	10
2.6	Heuristic-Guided Planning	12
2.7	Sampling Distribution and Exploration Properties	12
2.8	Greedy Connect Heuristic	13
2.9	Probabilistic Completeness and Failure Recovery	13
2.10	Summary	14
3	Related Works	15
3.1	Rapidly-Exploring Random Trees	16
3.2	Bidirectional Rapidly-Exploring Random Trees	16
3.3	Spatial Offset Sampling-RRT* (SOF-RRT*)	17

3.4	Summary and Research Gap	17
4	Proposed Solution	19
4.1	Background	19
4.1.1	Problem Definition	20
4.1.2	Bidirectional RRT (RRT-Connect)	20
4.1.3	The EXTEND Function	21
4.1.4	The CONNECT Function	22
4.2	Enhanced Bidirectional RRT (EBiRRT) Algorithm	23
4.2.1	Planner Overview	23
4.2.2	Heuristic Function Definition	24
4.2.3	Sampling Strategies	25
4.2.4	Adaptive Bias Probability	26
4.2.5	Heuristic Guide-Pair Selection	27
4.2.6	EBiRRT Variants: Case 1 and Case 2	30
4.2.6.1	EBiRRT1 (Case 1): Target-biased with Global Sampling	31
4.2.6.2	EBiRRT2 (Case 2): Target-biased with Circle- based Sampling	31
4.2.7	Adaptive Probability Update	32
4.3	SOF-RRT Algorithm	33
4.3.1	Motivation	33
4.3.2	Simulated LiDAR-Based Spatial Partitioning	34
4.3.3	Probability Weight Assignment	35
4.3.4	Selection of the SOF Reference Node	36
4.3.5	Integration into EBiRRT	37
4.3.6	Artificial Potential Field Bias-Goal (AFBG) Steering Strategy	37
4.3.7	Goal-Attraction Bias	39

4.3.8	Obstacle-Tangential Bias	40
4.3.9	Full AFBG Steering Rule	42
4.3.10	Integration into EBiRRT (SOF-EBiRRT)	42
5	Experimental Results	46
5.1	Environment Setup	46
5.1.1	Environment setup	46
5.1.2	Visualization Settings	47
5.1.3	Parameter Settings	50
5.1.4	Testing Scenarios	51
5.2	Result Visualization	52
5.2.1	Case 1: Simple Environment (10% Obstacles)	52
5.2.2	Case 2: Typical Environment (20% Obstacles)	53
5.2.3	Case 3: Complex Environment (40% Obstacles)	53
5.3	Result Analysis	54
5.3.1	Performance in Simple and Typical Environments	54
5.3.2	Performance in Complex Environments	56
6	Conclusion	57
6.1	Summary	57
6.2	Remarks	58
6.3	Limitations	59
6.4	Future Work	59
	References	61

Chapter 1

Introduction

In chapter 1, the overview, objectives and goals of the research's project are illustrated. The outline of the report is also presented.

1.1 Motivation

Motion planning in complex, cluttered environments remains a pivotal challenge in autonomous robotics. While bidirectional sampling-based algorithms (e.g., BiRRT) are industry standards due to their probabilistic completeness, they exhibit fundamental inefficiencies when deployed in scenarios with high obstacle densities.



Figure 1.1: Illustration of wasted exploration in a cluttered environment

The limitations of standard BiRRT-based planners can be attributed to three main factors:

1. **Blind Exploration:** Standard BiRRT relies heavily on uniform random sampling. This "blind" approach ignores the geometric relationship between the start and goal trees, wasting computational resources on expanding into irrelevant regions rather than converging towards a solution.
2. **Lack of Directional Guidance:** Traditional approaches lack a mechanism to identify and prioritize node pairs that are well-positioned for connection. Without this guidance, the planner often engages in redundant exploration instead of exploiting promising opportunities to bridge the gap between the two trees.
3. **Static Sampling Strategy:** Conventional planners employ a sampling strategy that does not differentiate between global exploration and precise connection. The inability to dynamically constrain the sampling region—such as focusing on the corridor between trees—reduces the probability of successfully navigating narrow passages.

These challenges motivate the development of the **EBi-RRT** (Enhanced Bidirectional RRT) framework. By introducing a *target-biased expansion strategy* and an *adaptive probability mechanism*, this work aims to bridge the gap between exploration efficiency and connection robustness. The proposed planner dynamically shifts between aggressive geometric sampling and robust random exploration, significantly reducing planning time and improving success rates in complex environments.

1.2 Goals

The primary goal of this project is to overcome the limitations of traditional bidirectional planners in cluttered environments by developing an enhanced frame-

work named **EBi-RRT**. The core objective is to achieve a superior balance between exploration speed and path optimality through a novel *Target-biased Expansion Strategy*.

Specifically, the research objectives are condensed as follows:

- **Heuristic-Driven Adaptive Expansion:** To formulate a connectivity-oriented heuristic function combined with an adaptive bias probability mechanism. This aims to intelligently regulate the trade-off between global exploration and local exploitation, allowing the planner to prioritize promising connections while avoiding stagnation in dead-end.
- **Investigation of Sampling Strategies:** To propose and evaluate distinct geometric sampling constraints specifically comparing *Target-biased Global Sampling* (EBi-RRT1) and *Target-biased Circle Sampling* (EBi-RRT2)—to determine the most effective strategy for navigating narrow passages and complex topologies.
- **Optimization and Validation:** To implement algorithmic optimizations (e.g., spatial indexing) for real-time performance and to conduct a comprehensive experimental evaluation. The proposed solution will be benchmarked against standard baselines (BiRRT) across environments of varying complexity to validate improvements in success rate, search efficiency, and path quality.

1.3 Scope

This project focuses on sampling-based motion planning for autonomous mobile robots operating in static and fully known environments. The workspace is represented using occupancy grids, where obstacles are modeled as fixed regions and the free space is assumed to be accurately known in advance.

The proposed algorithm is designed for point-mass or holonomic robot models, where the robot can move freely in the configuration space without kinematic

constraints. The main objective of the planner is to compute collision-free paths that connect a given start configuration to a goal configuration in a continuous configuration space. Issues related to robot dynamics, such as velocity limits, acceleration bounds, or non-holonomic constraints, are not considered in this work.

The experimental evaluation is conducted in two-dimensional and simplified three-dimensional environments. All environments are static, meaning that obstacle positions do not change during the planning process. Dynamic obstacles, moving agents, and real-time environment updates are outside the scope of this project. In addition, the planner assumes perfect sensing and localization and does not address uncertainty in perception, mapping, or actuation.

The scope of this study is limited to offline or quasi-offline planning in simulation. Although the proposed method is designed with computational efficiency in mind, real-time execution on physical robotic platforms and integration with low-level control systems are not addressed. The planner is evaluated primarily on planning success rate, search efficiency, and path quality rather than execution robustness.

Within this defined scope, the project emphasizes algorithmic design and analysis. The main contributions lie in heuristic-guided sampling, adaptive bias control, and bidirectional planning strategies. The work also includes an optional local optimization step inspired by RRT* rewiring to improve path smoothness. Overall, the project aims to provide a practical and extensible BRRT-based planning framework for complex, cluttered environments under controlled assumptions.

1.4 Report Structure

This capstone project report is organized into six chapters as follows:

- **Chapter 1: Introduction** presents the motivation, research goals, project scope, and main contributions of the study. It also provides an overview of the report structure.
- **Chapter 2: Theoretical Background** provides the fundamental concepts required for the project, including configuration space, sampling-based motion planning, Rapidly-exploring Random Trees (RRT), bidirectional RRT variants, RRT*, heuristic-guided planning, sampling strategies, steering methods, and failure recovery.
- **Chapter 3: Related Works** reviews existing sampling-based path planning methods, including RRT, bidirectional RRT approaches, and SOF-RRT*. This chapter discusses their advantages and limitations, thereby identifying the research gap addressed by the proposed method.
- **Chapter 4: Proposed Solution** presents the proposed heuristic-guided BRRT-based planning framework. It introduces the baseline formulation, the EBiRRT algorithm with two variants, and the SOF-EBiRRT extension with advanced sampling and steering strategies.
- **Chapter 5: Experimental Results** describes the experimental setup and testing scenarios, visualizes the planning results, and provides a quantitative analysis of BiRRT, EBiRRT1, EBiRRT2, and SOF-EBiRRT in environments with different obstacle densities.
- **Chapter 6: Conclusion** summarizes the main contributions and key experimental findings of the project, emphasizing the effectiveness and trade-offs of the proposed SOF-EBiRRT algorithm.

Chapter 2

Theoretical Background

In Chapter 2, the preliminary knowledge and fundamental concepts related to sampling-based motion planning are presented.

2.1 Overview of Path Planning Algorithms

Path planning is the fundamental process of determining a collision-free path from a starting position to a goal position within a specific environment. Typical applications range from autonomous vehicles operating in structured warehouse layouts to complex robotic manipulation tasks. Approaches to solving this problem are generally categorized into two main classes: graph-based search algorithms and sampling-based algorithms.

2.1.1 Graph-based Approaches

Graph-based methods operate by discretizing the configuration space and systematically searching for a solution. Prominent algorithms in this category include Dijkstra's algorithm, A*, and D*.

- **Completeness and Optimality:** These algorithms are theoretically complete and optimal, meaning they are guaranteed to find an optimal solution whenever one exists.

- **Limitations:** The primary drawback of graph-based approaches is that the search cost depends significantly on the resolution, dimensionality, and size of the map. As the complexity of the environment increases, these methods often suffer from the "curse of dimensionality," making them computationally expensive for high-dimensional configuration spaces.

2.1.2 Sampling-based Approaches

To address the computational challenges of graph-based methods in large-scale environments, sampling-based algorithms were introduced. Instead of exploring all possible cells in a grid, these algorithms examine sample points generated randomly or heuristically within the configuration space.

- **Probabilistic Roadmaps (PRM):** Proposed by Kavraki et al., PRM preprocesses the environment to build a graph (roadmap) of random samples, upon which graph-based searching is applied. PRM is particularly suitable for multi-query path planning, where multiple queries are invoked within the same static environment.
- **Rapidly-exploring Random Trees (RRT):** Introduced by LaValle, RRT is designed for single-query path planning. It incrementally builds a tree rooted at the start configuration by extending toward random samples. RRT is preferred for its simple implementation, low cost, and ability to handle high-dimensional configurations efficiently. The standard RRT employs a uniform sampling strategy, which ensures the property of probabilistic completeness.

While standard sampling-based algorithms improve search efficiency, they may sample unnecessary spaces or produce suboptimal paths. Consequently, numerous variants have been proposed to optimize specific performance metrics:

- **Path Optimization:** *RRT** improves upon RRT by rewiring tree connections to asymptotically approach the optimal path length. *Informed RRT**

further speeds up this process by restricting the sampling area to an ellipsoidal subset of the configuration space.

- **Heuristic and Goal Biasing:** To accelerate convergence, strategies such as goal-biased sampling introduce a probability of sampling the goal configuration directly.
- **Bidirectional Search:** Drawing on bidirectional search concepts, algorithms like *RRT-Connect* construct two trees simultaneously—one from the start and one from the goal—to reduce planning time significantly.

The challenge in path planning remains balancing success rate, search time, computational cost, and path length. The methods discussed in this chapter form the theoretical basis for the proposed enhancements presented in this thesis.

2.2 Configuration Space and Path Planning

Robot motion planning is often described using the configuration space (C-space). The configuration space includes all possible positions and orientations of a robot. Each point in the C-space represents one specific configuration of the robot [1], [2].

The configuration space is divided into two parts. The obstacle space $\mathcal{C}_{obs}$ contains configurations where the robot collides with obstacles. The free space $\mathcal{C}_{free}$ contains all configurations where the robot can move without collision [1].

Given a start configuration $s_0 \in \mathcal{C}_{free}$ and a goal configuration $t_0 \in \mathcal{C}_{free}$, the goal of path planning is to find a continuous path $\pi : [0, 1] \rightarrow \mathcal{C}_{free}$ such that $\pi(0) = s_0$ and $\pi(1) = t_0$ [2].

In many real environments, the configuration space is large and complex. Because of this, it is difficult to explicitly build the free space $\mathcal{C}_{free}$. For this reason, sampling-based motion planning algorithms are often used. These methods explore the configuration space by random sampling instead of full construction [2], [3].

2.3 Rapidly-exploring Random Trees (RRT)

Rapidly-exploring Random Trees (RRT) are sampling-based motion planning algorithms designed to efficiently explore high-dimensional configuration spaces. An RRT incrementally builds a tree rooted at an initial configuration by repeatedly sampling random configurations and extending the tree toward them.

At each iteration, a random configuration q_{rand} is sampled from the configuration space. The nearest vertex q_{near} in the tree is selected using a distance metric, and a new configuration q_{new} is generated by moving from q_{near} toward q_{rand} with a fixed incremental step size. If the motion segment is collision-free, q_{new} is added to the tree.

RRTs are probabilistically complete and exhibit a natural bias toward unexplored regions of the configuration space due to their Voronoi-based expansion behavior [4].

2.4 Fundamentals of Bidirectional RRT

Bidirectional RRT (BiRRT) methods enhance the exploration process by growing two trees simultaneously: one rooted at the initial configuration (q_{start}) and the other at the goal configuration (q_{goal}) [4], [5]. The fundamental objective is to reduce planning time by enabling the two trees to meet in the configuration space.

Among bidirectional variants, RRT-Connect [6] is widely adopted for single-query planning due to its introduction of a greedy heuristic. Unlike the standard RRT which extends by a single step ϵ , RRT-Connect employs two distinct operations:

- **EXTEND:** The tree extends a single step toward a random sample q_{rand} .
- **CONNECT:** If the extension is successful, the opposite tree aggressively attempts to reach the newly added node by repeatedly applying the step size

ϵ until a collision occurs or the connection is established [6].

In each iteration, the roles of the two trees are swapped to ensure balanced expansion. This greedy connection strategy allows the planner to rapidly bridge the gap between the start and goal regions in open spaces.

2.5 RRT* and Path Optimization

RRT* is an asymptotically optimal extension of the standard RRT algorithm. While RRT efficiently explores the high-dimensional state space to find a feasible path, it typically converges to a suboptimal solution with probability one [7]. RRT* addresses this limitation by introducing a cost function and managing the tree topology to ensure convergence to the optimal solution as the number of samples approaches infinity.

Optimal Motion Planning Formulation

The objective of RRT* is to find a path $\sigma : [0, T] \rightarrow X_{free}$ connecting an initial state x_{init} to a goal region X_{goal} such that it minimizes a specific cost function $c(\sigma)$ [7]. The algorithm maintains a tree $\mathcal{T} = (V, E)$, where each node $z \in V$ is associated with a cost value $Cost(z)$, representing the accumulated cost of the path from the root to z .

Tree Expansion and Parent Selection

Unlike RRT, which simply connects the new sample z_{new} to the nearest node $z_{nearest}$, RRT* searches for the best parent within a defined neighborhood to minimize the accumulated cost. Let Z_{near} be the set of nodes in the tree within a specific radius of z_{new} . The algorithm selects the parent node $z_{min} \in Z_{near}$ that minimizes the total cost to reach z_{new} :

$$z_{min} = \underset{z \in Z_{near}}{\operatorname{argmin}} (Cost(z) + c(z, z_{new})) \quad (2.1)$$

where $c(z, z_{new})$ denotes the cost of the trajectory between z and z_{new} (e.g.,

Euclidean distance or path length) [7]. Once z_{min} is chosen, z_{new} is added to the tree with an edge (z_{min}, z_{new}) .

Rewiring Process

The most critical innovation of RRT* is the “Rewiring” step, which restructures the tree to improve existing paths. After inserting z_{new} , the algorithm checks every neighbor $z_{near} \in Z_{near} \setminus \{z_{min}\}$ to determine if reaching z_{near} via z_{new} is more efficient than its current parent path.

The rewiring condition is satisfied if:

$$Cost(z_{new}) + c(z_{new}, z_{near}) < Cost(z_{near}) \quad (2.2)$$

If this inequality holds and the path connecting z_{new} to z_{near} is collision-free, the algorithm removes the edge between z_{near} and its old parent, and adds a new edge (z_{new}, z_{near}) . This effectively lowers the cost for z_{near} and all its descendants in the tree [7].

Asymptotic Optimality

Through the iterative application of optimal parent selection and local rewiring, RRT* guarantees asymptotic optimality. Karaman and Frazzoli proved that the probability of finding the optimal solution converges to one as the number of samples $N \rightarrow \infty$:

$$P\left(\lim_{N \rightarrow \infty} Cost(Path_N) = Cost(Path^*)\right) = 1 \quad (2.3)$$

This property makes RRT* highly suitable for real-time applications where solution quality is paramount, although it incurs a higher computational cost per iteration compared to the standard RRT due to the nearest-neighbor search and collision checks involved in rewiring [7].

2.6 Heuristic-Guided Planning

Heuristic-guided planning introduces additional guidance to sampling-based algorithms by assigning heuristic scores to nodes or node pairs. These scores provide an estimate of how promising a node is for reaching the goal, typically based on geometric distance or cost.

In bidirectional planning methods, heuristic functions are often used to evaluate pairs of nodes from the start tree and the goal tree. The heuristic value estimates how likely it is that the two nodes can be connected by a collision-free path[6], [7].

During planning, node pairs with lower heuristic values are selected as guide-pairs. These pairs are considered more promising, so the planner focuses sampling and tree extension in regions where a valid path is more likely to be found. This strategy can significantly reduce planning time compared to unguided sampling [8], [9].

However, the performance of heuristic-guided planning depends strongly on the quality of the heuristic. If the heuristic is poorly designed, the planner may focus too much on unhelpful regions of the configuration space. This can cause biased exploration or stagnation, where the same ineffective guide pairs are selected many times without making progress [10], [11].

2.7 Sampling Distribution and Exploration Properties

RRT and RRT-Connect rely on random sampling to guide exploration of the configuration space. Although samples are typically drawn from a uniform distribution over the free configuration space, the distribution of RRT vertices evolves over time.

Kuffner and LaValle showed that the distribution of vertices generated by RRT

converges in probability to the sampling distribution, implying uniform coverage of the free space in the limit [6]. The *CONNECT* heuristic introduces additional vertices during aggressive extension, which further contribute to coverage without negatively affecting probabilistic completeness.

As a result, RRT-Connect maintains probabilistic completeness while significantly improving practical convergence speed.

2.8 Greedy Connect Heuristic

The *CONNECT* heuristic is a greedy extension strategy introduced in RRT-Connect to accelerate convergence. Unlike the standard *EXTEND* operation, which performs a single incremental step toward a target configuration, *CONNECT* repeatedly applies the *EXTEND* operation until the target is reached or further progress is blocked by obstacles.

This heuristic functions similarly to an artificial potential field, enabling rapid progress toward promising regions of the configuration space. However, unlike classical potential field methods, the greedy behavior in RRT-Connect is combined with the exploration properties of RRTs, reducing the risk of stagnation in dead-end [6].

The attractive movement created by the growing trees allows the planner to maintain both exploration and exploitation simultaneously.

2.9 Probabilistic Completeness and Failure Recovery

RRT-Connect is probabilistically complete, meaning that if a collision-free path exists, the probability that the planner finds a solution approaches one as the number of iterations increases [6].

Despite this theoretical guarantee, practical performance may degrade in highly cluttered environments or narrow passages. Repeated failed extensions may

occur when the greedy CONNECT heuristic encounters obstacles.

These observations motivate the introduction of robustness mechanisms such as failure tracking, adaptive exploration strategies, and dead-end escape behaviors, which are further developed in the proposed algorithm.

2.10 Summary

This chapter presented the theoretical foundations of sampling-based motion planning relevant to this project. The concepts of configuration space, RRT, bidirectional planning, heuristic guidance, informed sampling, and robustness mechanisms provide the basis for the proposed heuristic-guided BRRT-based algorithm described in subsequent chapters.

Chapter 3

Related Works

Sampling-based motion planning algorithms have been extensively studied and widely applied to robot navigation problems in complex and high-dimensional environments. Among them, Rapidly-exploring Random Trees (RRT) and their bidirectional variants have demonstrated strong performance due to their probabilistic completeness and scalability. However, despite their effectiveness, classical RRT-based planners often suffer from inefficient sampling, stagnation in narrow passages, and repeated unsuccessful connection attempts in cluttered environments. As a result, numerous extensions and improvements have been proposed to enhance their exploration efficiency, convergence speed, and robustness.

In this context, this chapter reviews several related works that address these challenges from different perspectives, including bidirectional planning, heuristic-guided exploration, informed and obstacle-aware sampling strategies, and path optimization techniques. The chapter provides a comprehensive overview of these methods, discusses their advantages and limitations, and establishes the motivation for the proposed heuristic-guided BRRT-based algorithm presented in this thesis.

3.1 Rapidly-Exploring Random Trees

Rapidly-exploring Random Trees (RRT) are a class of sampling-based motion planning algorithms designed to efficiently explore high-dimensional configuration spaces. Since their introduction by LaValle [4], RRTs have been widely adopted due to their probabilistic completeness and scalability. The core idea of RRT is to incrementally grow a tree rooted at the start configuration by repeatedly sampling random states and extending the tree toward these samples.

This exploration strategy naturally biases the tree growth toward unexplored regions of the configuration space, making RRT particularly suitable for complex planning problems. However, classical RRT does not guarantee path optimality and often produces suboptimal paths with unnecessary detours. Moreover, uniform random sampling may lead to inefficient exploration in cluttered environments, where many samples fail to contribute to meaningful progress toward the goal.

3.2 Bidirectional Rapidly-Exploring Random Trees

To address the slow convergence of unidirectional RRTs, Kuffner and LaValle proposed RRT-Connect, a bidirectional variant that significantly accelerates the search process [6]. As detailed in Section 2.4, RRT-Connect leverages a greedy heuristic to aggressively bridge the gap between the start and goal trees. By introducing the swap mechanism and the *CONNECT* strategy, this approach has become a standard baseline in sampling-based motion planning due to its probabilistic completeness and superior performance in uncluttered environments [6].

Limitations: Despite its effectiveness in open spaces, the standard RRT-Connect algorithm exhibits notable inefficiencies in cluttered environments or narrow passages. Moreover, the lack of directional guidance means the planner may waste computational resources exploring irrelevant regions rather than converg-

ing toward the connection point.

3.3 Spatial Offset Sampling-RRT* (SOF-RRT*)

RRT* extends the classical RRT framework by introducing a rewiring mechanism that guarantees asymptotic optimality [7]. However, in complex environments, the algorithm often suffers from slow convergence and high initial path costs. These issues stem from the low probability of effective sampling, caused by invalid tree expansions and redundant sampling in cluttered areas [12].

To address these limitations, Yu et al. proposed the SOF-RRT* algorithm [12]. A key innovation of SOF-RRT* is the **spatial probability-weight sampling strategy**. Instead of the uniform sampling used in standard RRT*, this method simulates a lidar sensor at a selected node to emit ranging rays, thereby dividing the local space into sector blocks. Weights are assigned to these blocks based on ray lengths (representing obstacle proximity), effectively biasing sampling towards larger collision-free regions to increase the success rate [12].

Additionally, SOF-RRT* enhances tree expansion through two biases:

- **Target Bias:** This mechanism guides tree growth towards the goal configuration, reduce the disadvantages of random exploration and improving expansion efficiency [12].
- **Obstacle Tangential Bias:** When a node is proximal to an obstacle, this strategy adjusts the expansion direction tangentially to the obstacle’s boundary. This mechanism enables the tree to bypass obstacles smoothly, converting potentially invalid expansions into effective ones [12].

3.4 Summary and Research Gap

In summary, existing research has explored bidirectional planning, heuristics biasing, and obstacle-aware sampling from different perspectives. While bidirectional RRT methods improve convergence speed, they often lack effective

guidance in cluttered environments. Heuristic and informed sampling strategies enhance exploration efficiency but are mainly designed for single-tree planners. Obstacle-aware approaches further improve robustness but increase computational complexity.

These limitations motivate the development of a heuristic-guided bidirectional RRT framework that balances exploration efficiency, connection robustness, and computational cost.

Chapter 4

Proposed Solution

As discussed in Chapter 3, classical bidirectional sampling-based motion planning algorithms such as RRT-Connect exhibit limitations in cluttered environments, including inefficient sampling, stagnation in narrow passages, and repeated unsuccessful connection attempts. To address these challenges, this chapter presents the proposed heuristic-guided BRRT-based planning framework. The proposed solution is organized into three main parts: (1) background and baseline algorithms, (2) the EBiRRT algorithm with Case 1 and Case 2 variants, and (3) the SOF-RRT algorithm with advanced sampling and steering strategies. Together, these components aim to improve exploration and exploitation efficiency, convergence reliability, and path quality in complex environments.

4.1 Background

To evaluate the effectiveness of the proposed heuristic-guided BRRT-based algorithm, we first establish the theoretical foundation and the baseline algorithms used for comparison. This section details the problem definition and the standard Bidirectional RRT (BiRRT) strategy, specifically the RRT-Connect algorithm.

4.1.1 Problem Definition

Similarly to , we define the global path planning problem in a configuration space $C \subseteq \mathbb{R}^d$, where $d \in \mathbb{N}$ and $d \geq 2$. Let $C_{free} \subseteq C$ represent the obstacle-free region, defined as $C_{free} = C \setminus C_{obs}$, where C_{obs} denotes the set of obstacle regions.

Given a starting configuration $q_{start} \in C_{free}$ and a goal configuration $q_{goal} \in C_{free}$, the path planning problem can be modeled as obtaining a collision-free path from q_{start} to q_{goal} .

4.1.2 Bidirectional RRT (RRT-Connect)

In this work, we adopt the BiRRT strategy [6], also known as RRT-Connect. Unlike the standard RRT which grows a single tree to explore the state space, BiRRT maintains two trees simultaneously: tree S at root q_{start} and tree T at root q_{goal} . The objective is to advance these two trees toward each other to find a connection in the free space, which significantly improves the convergence rate compared to single-tree expansion.

The main logic is presented in **Algorithm 1** below. At each iteration, the planner samples a random configuration q_{rand} and attempts to extend tree S toward it using the EXTEND function. If the extension is successful (i.e., not Trapped), the planner then attempts to aggressively connect tree T to the newly added node of S using the CONNECT function. After each iteration, the roles of tree S and tree T are swapped to ensure balanced expansion from both ends.

Algorithm 1 RRT_CONNECT_PLANNER(q_{start}, q_{goal})

```
1:  $S.init(q_{start}); T.init(q_{goal});$ 
2: for  $k = 1$  to  $K$  do
3:    $q_{rand} \leftarrow \text{RANDOM\_CONFIG}();$ 
4:    $q_{near\_S} \leftarrow \text{NEAREST\_NEIGHBOR}(q_{rand}, S);$ 
5:   if not ( $\text{EXTEND}(S, q_{rand}, q_{near\_S}, q_{new\_S}) = \text{Trapped}$ ) then
6:      $q_{near\_T} \leftarrow \text{NEAREST\_NEIGHBOR}(q_{new\_S}, T);$ 
7:     if  $\text{CONNECT}(T, q_{new\_S}, q_{near\_T}) = \text{Reached}$  then
8:       return  $\text{PATH}(S, T);$ 
9:     end if
10:  end if
11:   $\text{SWAP}(S, T)$ 
12: end for
13: return Failure
```

4.1.3 The EXTEND Function

The EXTEND function (Algorithm 2) is responsible for incrementally expanding the tree. It attempts to add a new node q_{new} to the current tree by taking a fixed step size ϵ from the nearest node q_{near} in the direction of the target sample q_{rand} . Before adding the node, the function checks if the path segment from q_{near} to q_{new} lies entirely within C_{free} .

There are three possible return statuses:

- **Reached:** The distance to the target q_{rand} is less than the step size ϵ , meaning the target is reached directly ($q_{new} = q_{rand}$).
- **Advanced:** A new node q_{new} is successfully created towards q_{rand} , but the target has not yet been reached ($q_{new} \neq q_{rand}$).
- **Trapped:** The expansion is blocked by an obstacle, and no new node is created.

Algorithm 2 EXTEND($curTree, q_{rand}, q_{near}, q_{new}$)

```
1: if NEW_CONFIG( $q_{rand}, q_{near}, q_{new}$ ) then
2:    $curTree.add\_vertex(q_{new})$ ;
3:    $curTree.add\_edge(q_{near}, q_{new})$ ;
4:   if  $q_{new} = q_{rand}$  then
5:     return Reached;
6:   else
7:     return Advanced;
8:   end if
9: end if
10: return Trapped;
```

4.1.4 The CONNECT Function

The CONNECT function (Algorithm 3) applies a greedy heuristic to bridge the gap between the two trees. Unlike EXTEND, which takes a single step, CONNECT repeatedly invokes the EXTEND function to advance the tree toward a target point (typically the newly added node from the opposite tree). This process continues until either the target is reached (successfully merging the trees) or the extension becomes blocked by an obstacle.

Algorithm 3 CONNECT($curTree, q_{rand}, q_{near}$)

```
1: repeat
2:    $m \leftarrow \text{EXTEND}(curTree, q_{rand}, q_{near}, q_{new})$ ;
3:    $q_{near} \leftarrow q_{new}$ ;
4: until not ( $m = \text{Advanced}$ )
5: return  $m$ 
```

The interaction between these operations is visualized in Figure 4.1. The EXTEND step makes incremental progress to maintain exploration coverage, while CONNECT exploits the known free space to aggressively connect the trees.

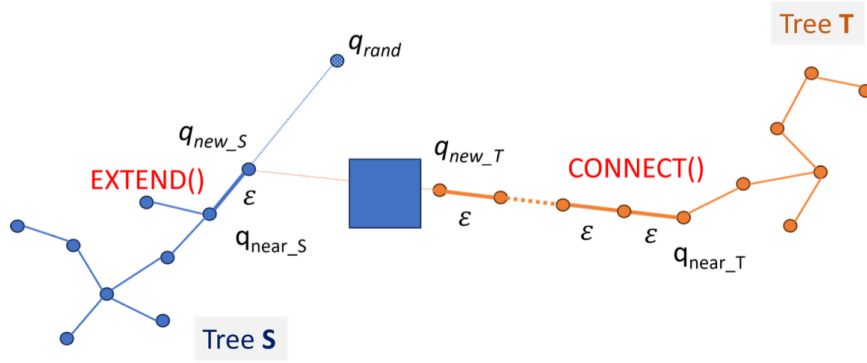


Figure 4.1: The operation of EXTEND and CONNECT functions. Tree S extends incrementally towards a random sample, while Tree T attempts to connect greedily to the new node of S.

4.2 Enhanced Bidirectional RRT (EBiRRT) Algorithm

This section presents the Enhanced Bidirectional RRT (EBiRRT) algorithm, which introduces heuristic-guided expansion to improve upon the baseline BiRRT approach. The algorithm is implemented in two variants: EBiRRT1 (Case 1) and EBiRRT2 (Case 2), which differ in their sampling strategies.

4.2.1 Planner Overview

The proposed planner, referred to as **EBiRRT**, is a bidirectional sampling-based motion planning algorithm designed to handle cluttered and narrow environments. The planner simultaneously grows two trees: one from the start configuration and one from the goal configuration. These two trees are expanded iteratively and attempt to connect in the configuration space.

At each iteration, the planner probabilistically chooses between two expansion modes: heuristic-guided expansion and uninformed exploratory sampling. In the heuristic-guided mode, a guide pair is selected from the two trees to direct

the growth toward promising regions. In the exploratory mode, random sampling is used to maintain coverage of the configuration space and avoid local stagnation.

By alternating between these two modes, the planner balances directed search and global exploration, which improves robustness and convergence speed while preserving the structure of a bidirectional RRT-based algorithm.

4.2.2 Heuristic Function Definition

To further improve BiRRT's efficiency, we introduce a heuristic that prioritizes the expansion of branches closer to the target tree. We define a heuristic function to evaluate the potential for expanding the tree from a node $s_i \in S$ to a node $t_j \in T$ as follows:

$$h(s_i, t_j) = \alpha \cdot d(s_i, t_j) + \beta \cdot d(s_i, t_0) + \gamma \cdot d(t_j, s_0) \quad (4.1)$$

where:

- s_0 and t_0 represent the root nodes of trees S and T , respectively.
- $d(\cdot, \cdot)$ denotes the Euclidean distance between two points.
- $\alpha, \beta, \gamma \in \mathbb{R}^+$ are weighting coefficients used to adjust the influence of each term.

The proposed function combines the distance between the candidate nodes (s_i, t_j) with the distances from each node to the root of the opposite tree. A pair of nodes, denoted as $\{s_{guide}, t_{guide}\}$, is selected to guide the tree expansion if it minimizes this heuristic function. This approach prioritizes expansion at nodes that are more likely to achieve a successful connection between the two trees.

Figure 4.2 illustrates how the distances are calculated to determine the connection cost.

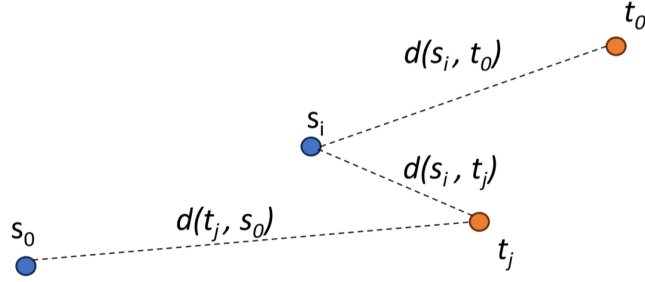


Figure 4.2: Illustration of heuristic function components: inter-node distance and distances to opposite roots.

4.2.3 Sampling Strategies

Once the guide pair $\{s_{guide}, t_{guide}\}$ is identified, the node $s_{guide} \in S$ is chosen to expand a new branch. A key distinction of our method compared to conventional bias-based strategies is that s_{guide} effectively serves as the q_{near} node directly, rather than searching for a nearest neighbor to a biased sample.

The planner then selects a random point q_{rand} to determine the expansion direction. We propose two strategies for sampling q_{rand} , as visualized in Figure 4.3:

- **Circle-based sampling (Strategy 1):** A random point q_{rand} is sampled within a circular region whose diameter is defined by the segment connecting s_{guide} and t_{guide} . This constrains the branch expansion to remain focused towards the target node t_{guide} .
- **Random sampling (Strategy 2):** A random point q_{rand} is sampled from the entire map. While the expansion origin is fixed at s_{guide} , the direction is determined globally, allowing the tree to potentially escape local traps or narrow passages.

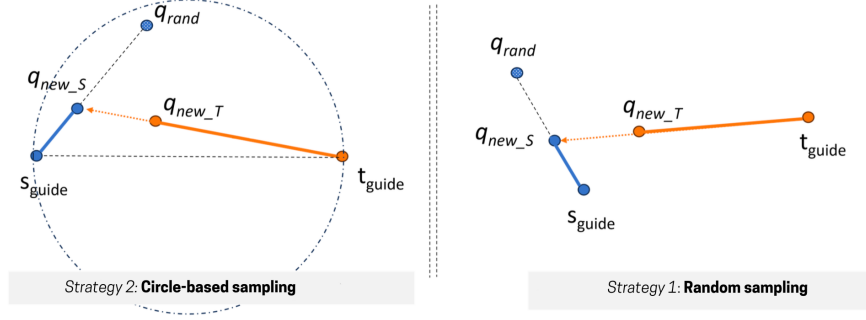


Figure 4.3: Illustration of the expansion strategies: Circle-based sampling (left) and Random sampling (right).

4.2.4 Adaptive Bias Probability

To balance between global exploration and target-oriented exploitation, the tree expansion strategy incorporates an adaptive probability mechanism. At each iteration, a random number $\rho \in [0, 1]$ is generated and compared with a dynamic bias threshold ρ_{bias} to determine the expansion mode:

- **Uniform expansion** ($\rho \geq \rho_{bias}$): The planner expands a new branch from the node closest to a randomly sampled point q_{rand} in the entire map. This mechanism maintains the probabilistic completeness of the algorithm.
- **Heuristic expansion** ($\rho < \rho_{bias}$): The planner selects a promising pair of nodes $\{s_{guide}, t_{guide}\}$ and expands from s_{guide} directly toward t_{guide} . This mechanism increases the likelihood of connecting the two trees.

The bias probability ρ_{bias} is dynamically adjusted based on the evolving heuristic distance. The algorithm prioritizes heuristic-guided growth in the early stages to quickly reduce the distance between trees, and gradually increases random expansions as the trees converge. The value of ρ_{bias} is calculated as:

$$\rho_{bias} = \rho_{init} \cdot e^{-\frac{\mu \cdot (h(q_{start}, q_{goal}) - h(s_{guide}, t_{guide}))}{h(q_{start}, q_{goal})}} \quad (4.2)$$

Where:

- ρ_{init} is the initial bias probability (typically set to 1).
- μ is a tuning factor that determines how strongly ρ_{bias} responds to changes in the heuristic value.
- $h(q_{start}, q_{goal})$ represents the initial heuristic distance.
- $h(s_{guide}, t_{guide})$ represents the current minimum heuristic distance between the two trees.

The relationship between the bias probability and the heuristic value is visualized in Figure 4.4. As the heuristic distance between the guide pair decreases (x-axis approaches 0), the bias probability ρ_{bias} decreases, shifting the focus towards uniform exploration to refine the path.

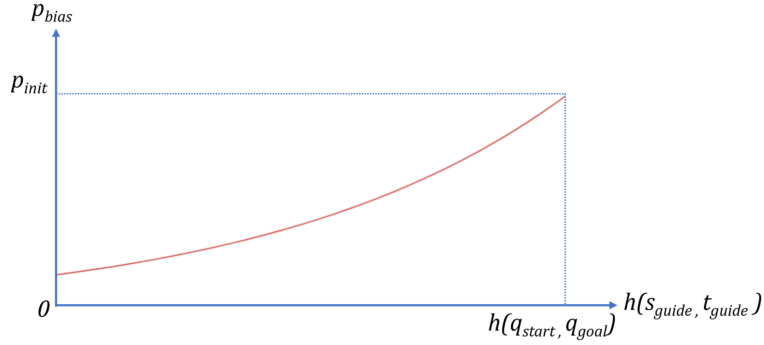


Figure 4.4: The relationship between ρ_{bias} and the heuristic value with $\mu = 2$.

4.2.5 Heuristic Guide-Pair Selection

The **FIND_GUIDE_PAIR** function is responsible for identifying the pair of nodes (s_{guide}, t_{guide}) that maximizes the likelihood of connecting the two trees. Unlike standard metrics that only consider the distance between nodes, our heuristic incorporates the relative progress of each node from its respective tree root.

For every pair of nodes $s_i \in S$ and $t_j \in T$, the heuristic cost $h(s_i, t_j)$ is calculated

as:

$$h(s_i, t_j) = \alpha \cdot \|s_i - t_j\| + \beta \cdot \|s_i - t_0\| + \gamma \cdot \|t_j - s_0\| \quad (4.3)$$

Where:

- $d(\cdot, \cdot)$ represents the Euclidean distance in the configuration space.
- s_0 and t_0 are the roots (q_{start} and q_{goal}) of the two trees.
- $\alpha, \beta, \gamma \in \mathbb{R}^+$ are weighting coefficients.

Physical Meaning of Parameters: The selection of α, β, γ is not arbitrary; each term serves a specific physical purpose in the planning dynamics:

- **The Attraction Term (α):** This coefficient governs the "greedy" nature of the connection. A large α prioritizes pairs (s_i, t_j) that are geometrically close to each other.
- **The Anchoring Terms (β and γ):** These terms represent the cost from the roots. They penalize nodes that are excessively far from their respective origins, implicitly encouraging the expansion frontier to move forward rather than spiraling in already explored regions.

Analysis of Parameter Sensitivity: The effectiveness of the heuristic function depends largely on the weights α, β, γ in Equation 4.3. These values control the balance between trying to connect the trees quickly and keeping the search near the root nodes. We examine two specific cases to show how these parameters affect the planner:

- *Case 1: Aggressive Connection ($\alpha \gg \beta, \gamma$):* In this scenario, the weight α is significantly larger than β and γ . Consequently, the planner prioritizes minimizing the distance between the two trees to connect them as rapidly as possible.

- **Advantage:** In open environments without obstacles, this strategy is optimal because the trees grow directly toward each other, reducing search time.
- **Disadvantage:** In complex environments, such as U-shaped obstacles, this approach can be problematic. The trees expand until they are geometrically close, separated only by a thin wall. Since the heuristic calculates a low cost based on distance, the planner continues attempting to connect through the wall. As a result, the trees become trapped in a local minimum and fail to explore the necessary path around the obstacle.
- *Case 2: Conservative Expansion ($\beta, \gamma \gg \alpha$):* In this case, the weights β and γ dominate the cost function. This means the heuristic value relies heavily on the distance from the node to the start or goal roots.
 - **Behavior:** The planner adopts a cautious strategy. Instead of extending toward the opposite tree, it prefers to add nodes that remain close to the tree roots (q_{start} or q_{goal}).
 - **Result:** This behavior prevents the trees from expanding too quickly into dead-ends, as seen in Case 1. However, it causes the search process to become slow. The trees grow outward gradually in a circular pattern rather than forming a directed line. Consequently, the time required for the two trees to meet increases significantly, reducing the overall convergence speed.

Conclusion: Therefore, identifying an appropriate balance between these behaviors is essential for the algorithm’s robustness. Through experimental validation, we determined that the ratio $\alpha = 7, \beta = 1, \gamma = 1$ provides the most effective balance. This configuration encourages rapid connection (high α) while maintaining sufficient ”anchoring” force (non-zero β, γ) to pull the search away from greedy sampling and facilitate valid obstacle avoidance. The algorithm

achieves this by evaluating all node pairs to select the one with the minimum heuristic cost.

Algorithm 4 FIND_GUIDE_PAIR(S, T)

```

1: Input: Tree  $S$ , Tree  $T$ 
2: Output: Pair  $\{s_{guide}, t_{guide}\}$  with minimum heuristic cost
3:  $min\_cost \leftarrow \infty$ 
4:  $\{s_{guide}, t_{guide}\} \leftarrow \{NULL, NULL\}$ 
5:  $s_{root} \leftarrow S.root()$ ,  $t_{root} \leftarrow T.root()$ 
6: for all  $s_i \in S$  do
7:   for all  $t_j \in T$  do
8:      $dist\_pair \leftarrow ||s_i - t_j||$ 
9:      $dist\_s\_to\_roots \leftarrow ||s_i - t_{root}||$ 
10:     $dist\_t\_to\_roots \leftarrow ||t_j - s_{root}||$ 
11:     $h\_val \leftarrow \alpha \cdot dist\_pair + \beta \cdot dist\_s\_to\_roots + \gamma \cdot dist\_t\_to\_roots$ 
12:    if  $h\_val < min\_cost$  then
13:       $min\_cost \leftarrow h\_val$ 
14:       $\{s_{guide}, t_{guide}\} \leftarrow \{s_i, t_j\}$ 
15:    end if
16:  end for
17: end for
18: return  $\{s_{guide}, t_{guide}\}$ 

```

4.2.6 EBiRRT Variants: Case 1 and Case 2

Based on the algorithmic implementation, both variants share the same underlying architecture for determining when to bias the search (using the adaptive probability p_{bias} calculated in Eq. 4.2). However, they differ fundamentally in how the expansion direction is determined during the *Exploitation* phase (i.e., when $random < p_{bias}$).

4.2.6.1 EBiRRT1 (Case 1): Target-biased with Global Sampling

As implemented in the `BRRT_Simple_Case1` class, this variant combines the heuristic node selection with global exploration.

- **Mechanism:** When the heuristic bias is triggered, the planner forces the expansion to originate from the optimal guide node s_{guide} (referred to as `selected_SI` in the code). However, the target sample q_{rand} remains a random configuration drawn from the entire free space (C_{free}).
- **Algorithmic Logic:**

$$q_{new} \leftarrow \text{STEER}(s_{guide}, q_{global_rand}, \epsilon) \quad (4.4)$$

- **Behavior:** This strategy anchors the tree growth to the most promising node but does not constrain the direction of growth. This allows the tree to "jump" to the best frontier node while still maintaining the ability to explore diverse directions, which is particularly useful for escaping local minima where the direct path to the goal is blocked.

4.2.6.2 EBiRRT2 (Case 2): Target-biased with Circle-based Sampling

As implemented in the `BRRT_Simple_Case2` class, this variant employs a stricter geometric constraint to enforce directional convergence.

- **Mechanism:** In the bias mode, the planner not only selects the optimal start node s_{guide} but also constrains the sampling region. It generates a sample q_{circle} strictly within a circular (or hyperspherical) region defined between the start guide node s_{guide} and the goal guide node t_{guide} .
- **Algorithmic Logic:**

$$q_{new} \leftarrow \text{STEER}(s_{guide}, q_{circle_sample}, \epsilon) \quad (4.5)$$

where q_{circle_sample} is generated by the function `randomPointInCircle`, creating a sampling corridor directly connecting the two trees.

- **Behavior:** This strategy is highly aggressive. It focuses all expansion efforts on the corridor between the two trees. This results in very fast convergence in open spaces or narrow passages that align with the connection vector, but it relies heavily on the adaptive bias mechanism to fallback to random sampling if the direct corridor is obstructed.

4.2.7 Adaptive Probability Update

The `UPDATE_PROBABILITY` function dynamically adjusts the sampling bias parameter ρ based on the convergence status of the planner. The goal is to prioritize heuristic-guided expansion in the early stages or when the trees are far apart, and gradually shift towards uniform exploration as the trees converge.

The probability ρ is updated at each iteration using the exponential decay formula:

$$\rho = \rho_{init} \cdot e^{-\frac{\mu \cdot (h_{initial} - h_{current})}{h_{initial}}} \quad (4.6)$$

where:

- $h_{initial} = h(q_{start}, q_{goal})$ is the heuristic cost at the start of the planning.
- $h_{current} = h(s_{guide}, t_{guide})$ is the heuristic cost of the current best pair.
- μ is a tuning factor controlling the decay rate.

Algorithm 5 UPDATE_PROBABILITY(s_{guide}, t_{guide})

```
1: Global:  $h_{initial}, \rho_{init}, \mu$ 
2:  $h_{current} \leftarrow \text{CALCULATE\_HEURISTIC}(s_{guide}, t_{guide})$ 
3: if  $h_{initial}$  is not set then
4:    $h_{initial} \leftarrow h_{current}$ 
5:   return  $\rho_{init}$ 
6: end if
7:  $ratio \leftarrow (h_{initial} - h_{current})/h_{initial}$ 
8:  $\rho \leftarrow \rho_{init} \cdot \exp(-\mu \cdot ratio)$ 
9: return  $\rho$ 
```

4.3 SOF-RRT Algorithm

This section presents the SOF-RRT (Spatial Offset Filter RRT) algorithm, which builds upon the EBiRRT framework by introducing advanced sampling strategies and steering mechanisms. The SOF-RRT algorithm integrates two key components: (1) the Spatial Offset (SOF) sampling strategy, which improves sampling efficiency in cluttered environments, and (2) the Artificial Potential Field Bias-Goal (AFBG) steering strategy, which provides goal-directed navigation while avoiding obstacles.

4.3.1 Motivation

The classical uniform random sampling strategy distributes sample points across the entire configuration space $\mathcal{C}$, which results in a large proportion of invalid samples falling inside obstacle regions, particularly in cluttered or narrow-passage environments. To address this inefficiency, we adopt the *Spatial Probability-Weight Sampling* strategy introduced in [12], which we refer to as the **SOF sampling strategy**.

In a standard RRT-based planner, the sampling step draws a random point $q_{rand} \sim$

$\mathcal{U}(\mathcal{C})$ uniformly over the entire map. In environments with dense obstacles or narrow corridors, the majority of these samples either lie inside obstacles or fall in regions from which the tree cannot expand effectively. This leads to a high ratio of *redundant* or *invalid* samples, degrading both the initial path quality and convergence speed.

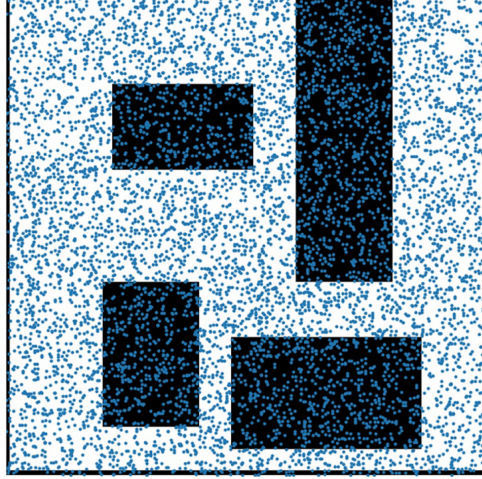


Figure 4.5: Illustration of redundant sampling under uniform random strategy: many samples fall inside obstacle regions (black), wasting iterations.

The SOF strategy addresses this by restricting the sampling domain to obstacle-free sectors around a chosen node and assigning higher probability weights to sectors with larger free-space extents.

4.3.2 Simulated LiDAR-Based Spatial Partitioning

Given a node q_{choose} selected from the current RRT tree $\mathcal{T}$, the surrounding free space is probed by simulating N_{block} rays emitted radially from q_{choose} , each with a maximum sensing range $maxR_{block}$. Each ray is truncated at the first obstacle boundary encountered, yielding a set of radial distances:

$$\mathbf{r} = \{r_1, r_2, \dots, r_{N_{block}}\}, \quad r_i \in (0, maxR_{block}], \quad (4.7)$$

where r_i is the free-space radius of the i -th angular sector. The full 2π angular range is then partitioned into N_{block} pie-shaped (sector) blocks $\{\mathcal{B}_1, \dots, \mathcal{B}_{N_{block}}\}$,

each $\mathcal{B}_i$ having its symmetry axis aligned with the i -th ray and its radius equal to r_i .

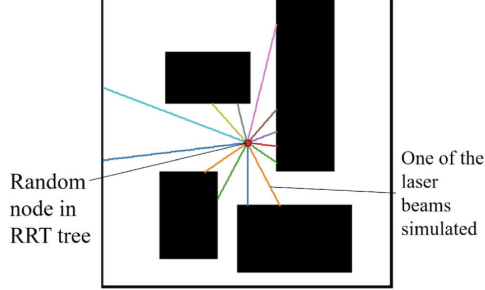


Figure 4.6: LiDAR-based spatial partitioning: rays are emitted radially from the selected node q_{choose} ; each ray is truncated at the nearest obstacle, defining the radius of its corresponding sector block.

4.3.3 Probability Weight Assignment

A weight is assigned to each sector block proportional to its free-space radius raised to the power of a *weight grade* parameter $WG \geq 0$:

$$SW = \sum_{i=1}^{N_{block}} r_i^{WG}, \quad W_i = \frac{r_i^{WG}}{SW}, \quad (4.8)$$

where SW denotes the total weight sum and W_i is the normalized sampling probability of sector $\mathcal{B}_i$. When $WG = 0$, this reduces to uniform sector sampling. For $WG > 0$, sectors with larger obstacle-free extents receive proportionally higher sampling probability, concentrating samples in the most accessible regions of the configuration space.

Once a sector $\mathcal{B}_i$ is selected according to $\{W_i\}$, a point q_{rand} is drawn uniformly within that sector, ensuring the sample lies in $\mathcal{C}_{free}$ by construction.

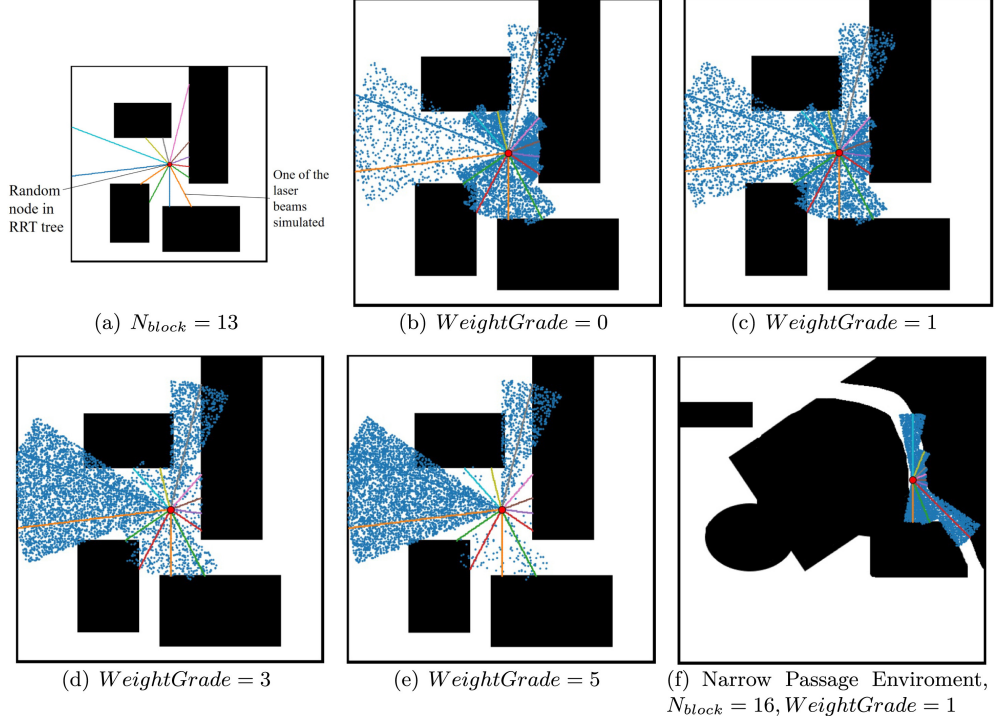


Figure 4.7: Effect of increasing weight grade WG on sampling distribution. Higher WG concentrates samples in sectors with larger free-space extents. In the narrow passage environment (f), samples are largely confined to the feasible corridor.

4.3.4 Selection of the SOF Reference Node

The node q_{choose} from which the LiDAR scan is launched depends on the active expansion mode. In the exploratory mode, where the planner must preserve broad coverage of the map, q_{choose} is selected using an ε -greedy policy:

- With probability ε : select a *random* node from $\mathcal{T}$ (exploration).
- With probability $1 - \varepsilon$: select the node in $\mathcal{T}$ *closest to the active target* q_{target} (exploitation).

The exploration factor ε decays geometrically over iterations:

$$\varepsilon \leftarrow \max(\varepsilon \cdot \gamma, \varepsilon_{floor}), \quad (4.9)$$

where $\gamma \in (0, 1)$ is the decay rate and ε_{floor} is a lower bound ensuring a minimum level of exploration throughout the planning process. This mechanism

allows the algorithm to explore broadly in early iterations and increasingly exploit proximity to the active target as planning progresses.

In the heuristic-guided mode, however, the reference node is not selected by the ε -greedy rule. After the guide pair (s_{guide}, t_{guide}) is obtained from Algorithm 4, the guide node belonging to the currently expanding tree is assigned directly to q_{choose} . Therefore, if the start tree $\mathcal{T}_S$ is being expanded, $q_{choose} = s_{guide}$; if the goal tree $\mathcal{T}_G$ is being expanded, $q_{choose} = t_{guide}$. This makes the SOF sampling step consistent with the EBiRRT expansion rule, where the guide node plays the role of the selected expansion anchor.

4.3.5 Integration into EBiRRT

In our EBiRRT framework, SOF sampling replaces the standard uniform sampling call in both Mechanism 1 (free-space sampling) and Mechanism 2 (heuristic-guided sampling between s_{guide} and t_{guide}). Concretely:

- In **Mechanism 1**, instead of drawing $q_{rand} \sim \mathcal{U}(\mathcal{C}_{free})$, we invoke the SOF procedure centered on q_{choose} to obtain a sample biased toward open space.
- In **Mechanism 2**, where the original EBiRRT uses the guide pair (s_{guide}, t_{guide}) to focus expansion, we apply SOF sampling locally around that guide pair. The guide node in the expanding tree is used directly as q_{choose} , and the resulting sample is used as the local expansion target.

The full SOF sampling procedure is summarized in Algorithm 6.

4.3.6 Artificial Potential Field Bias-Goal (AFBG) Steering Strategy

Once a sample q_{rand} has been drawn, the standard RRT steering step extends the nearest tree node $q_{nearest}$ by a fixed step δ in the direction of q_{rand} :

$$q_{new} = q_{nearest} + \delta \cdot \frac{q_{rand} - q_{nearest}}{\|q_{rand} - q_{nearest}\|}. \quad (4.10)$$

Algorithm 6 SOF Sampling: WEIGHTSAMPLE

Input: Tree nodes $\mathcal{T}$, map $\mathcal{M}$, mode m , optional guide node q_{guide} , target q_{target} , N_{block} , WG , ε , γ , ε_{floor} , P_{target}

Output: Sample point q_{rand} and reference node q_{choose}

```
1:  $\varepsilon \leftarrow \max(\varepsilon \cdot \gamma, \varepsilon_{floor})$ 
2: if  $m = \text{GUIDED}$  then
3:    $q_{choose} \leftarrow q_{guide}$  {For  $\mathcal{T}_S$ ,  $q_{guide} = s_{guide}$ ; for  $\mathcal{T}_G$ ,  $q_{guide} = t_{guide}$ }
4: else
5:   if  $\text{Rand}(1) < \varepsilon$  then
6:      $q_{choose} \leftarrow \text{RANDOMNODE}(\mathcal{T})$ 
7:   else
8:      $q_{choose} \leftarrow \text{GETNEARESTNODEOFTARGET}(\mathcal{T}, q_{target})$ 
9:   end if
10: end if
11: if  $\text{Rand}(1) \leq P_{target}$  then
12:    $q_{rand} \leftarrow q_{target}$ 
13:   return ( $q_{rand}, q_{choose}$ )
14: end if
15: Lines  $\leftarrow \text{GETBLOCKLINES}(q_{choose}, N_{block}, \mathcal{M})$ 
16:  $\{\mathcal{B}_i\} \leftarrow \text{GETBLOCKS}(\text{Lines})$ 
17:  $SW \leftarrow \sum_i r_i^{WG}$ 
18: Sample sector  $\mathcal{B}^*$  with probability  $W_i = \frac{r_i^{WG}}{SW}$ 
19:  $q_{rand} \leftarrow \text{UNIFORMSAMPLE}(\mathcal{B}^*)$ 
20: return ( $q_{rand}, q_{choose}$ )
```

While straightforward, this formulation is directionless in a global sense: the expansion is governed entirely by the random sample, leading to slow convergence and frequent collisions with obstacles when $q_{nearest}$ lies close to obstacle boundaries. Figure 4.8 illustrates the qualitative difference between unguided and AFBG-guided tree expansion.

To overcome these limitations, we adopt the *Artificial Potential Field Bias-Goal* (AFBG) steering strategy from [12], which modifies Equation (4.10) by using two structured bias terms: a *goal-attraction bias* and an *obstacle-tangential bias*.

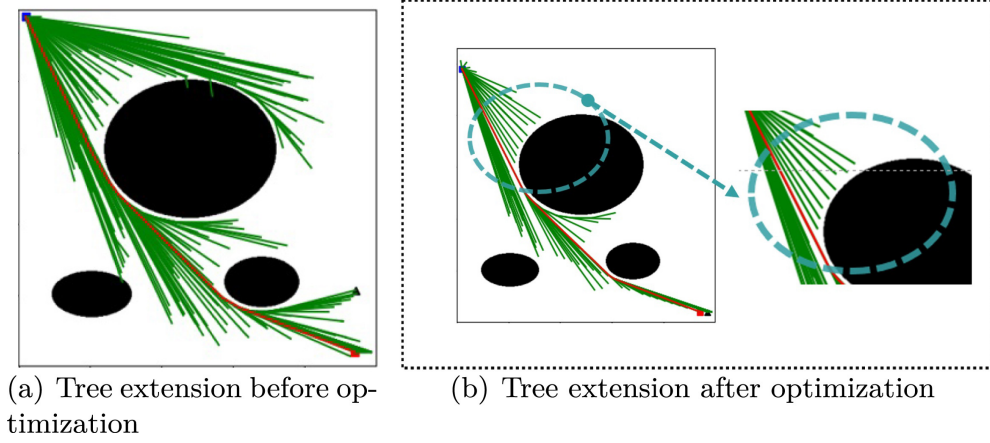


Figure 4.8: Comparison of tree expansion before and after applying the AFBG steering strategy. After optimization, branches are directed toward the goal and deflected tangentially along obstacle boundaries, resulting in fewer invalid expansions and a smoother tree structure.

4.3.7 Goal-Attraction Bias

Inspired by the attractive potential in classical Artificial Potential Field (APF) methods, a goal-bias vector is added to the base steering direction. This vector points from $q_{nearest}$ toward the goal q_{goal} , scaled by a goal-bias factor φ :

$$\varphi = \delta \cdot \sigma\left(\frac{d_{goal}}{d_{max}} \cdot 5\right), \quad (4.11)$$

where $d_{goal} = \|q_{nearest} - q_{goal}\|$ is the current distance to the goal, d_{max} is a normalization constant (e.g., the map diagonal), and $\sigma(\cdot)$ is the sigmoid function:

$$\sigma(x) = \frac{1}{1 + e^{-x}}. \quad (4.12)$$

The sigmoid constrains $\varphi \in (0, \delta)$, producing a smooth, bounded bias. The factor of 5 in the argument amplifies the sensitivity of φ to changes in normalized distance, exploiting the near-linear region of the sigmoid. When $q_{nearest}$ is far from the goal, $\varphi \approx \delta$, providing strong attraction; as the tree approaches q_{goal} , the bias naturally diminishes.

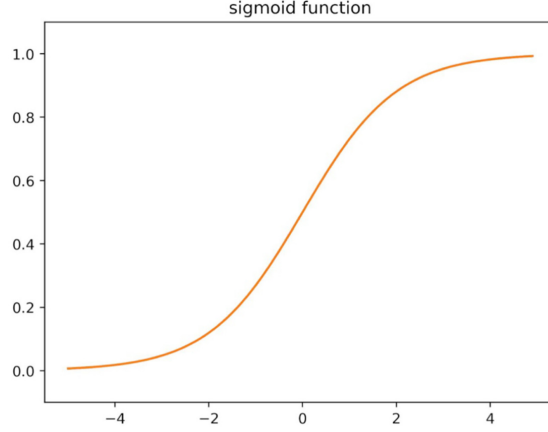


Figure 4.9: The sigmoid function used to compute the goal-bias factor φ and obstacle-repulsion factor η . The factor of 5 applied to the normalized input exploits the near-linear, most sensitive region of the curve.

4.3.8 Obstacle-Tangential Bias

When $q_{nearest}$ lies in close proximity to an obstacle (specifically, within a distance of 2δ), the standard extension direction may directly steer q_{new} into the obstacle. Rather than simply rejecting such expansions, the AFBG strategy deflects the new node tangentially along the obstacle boundary. An obstacle-repulsion factor η is computed as:

$$\eta = \delta \cdot \sigma\left(\frac{d_{obs}}{2\delta} \cdot 5\right), \quad (4.13)$$

where $d_{obs} = \text{dist}(q_{nearest}, \mathcal{C}_{obs})$ is the distance from $q_{nearest}$ to the nearest obstacle boundary, approximated via simulated ray scanning. When $q_{nearest}$ is very close to an obstacle, d_{obs} is small and η approaches δ , introducing a strong tangential deflection. As $q_{nearest}$ moves away from obstacles, $\eta \rightarrow 0$ and the obstacle bias vanishes.

Let $\mathcal{O}^*$ denote the obstacle with the smallest distance from $q_{nearest}$:

$$\mathcal{O}^* = \underset{\mathcal{O}_j \in \mathcal{C}_{obs}}{\operatorname{argmin}} \text{dist}(q_{nearest}, \mathcal{O}_j). \quad (4.14)$$

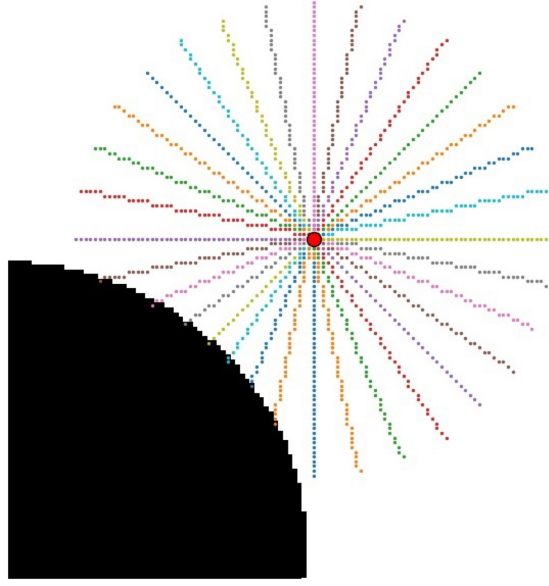


Figure 4.10: Approximation of $d_{obs} = \text{dist}(q_{nearest}, \mathcal{C}_{obs})$ using simulated laser scanning. Rays are cast outward from $q_{nearest}$; the minimum distance among all rays that encounter an obstacle boundary gives the nearest obstacle distance.

The closest obstacle-boundary point is then computed as:

$$q_{obs}^* = \underset{q \in \partial \mathcal{O}^*}{\text{argmin}} \|q - q_{nearest}\|, \quad d_{obs} = \|q_{obs}^* - q_{nearest}\|. \quad (4.15)$$

The obstacle-normal direction is the unit vector along the line connecting $q_{nearest}$ to the closest obstacle point:

$$\mathbf{n}_{obs} = \frac{q_{obs}^* - q_{nearest}}{\|q_{obs}^* - q_{nearest}\|}. \quad (4.16)$$

The tangential deflection is obtained by rotating this nearest-obstacle direction by 90° :

$$\mathbf{T}_{obs} = R_{\pm \frac{\pi}{2}} \mathbf{n}_{obs}, \quad \mathbf{T}_{obs} \perp \mathbf{n}_{obs}. \quad (4.17)$$

The sign of the tangential component (clockwise vs. counter-clockwise) is selected from the collision-free candidate that gives better progress toward the goal.

4.3.9 Full AFBG Steering Rule

Combining both biases, the AFBG-modified steering rule is:

$$q_{new} = \begin{cases} q_{nearest} + \delta \frac{q_{rand} - q_{nearest}}{\|q_{rand} - q_{nearest}\|} + \varphi \frac{q_{goal} - q_{nearest}}{\|q_{goal} - q_{nearest}\|} + \eta \mathbf{T}_{obs}, & \text{if } \text{dist}(q_{nearest}, \mathcal{C}_{obs}) < 2\delta, \\ q_{nearest} + \delta \frac{q_{rand} - q_{nearest}}{\|q_{rand} - q_{nearest}\|} + \varphi \frac{q_{goal} - q_{nearest}}{\|q_{goal} - q_{nearest}\|}, & \text{if } \text{dist}(q_{nearest}, \mathcal{C}_{obs}) \geq 2\delta. \end{cases} \quad (4.18)$$

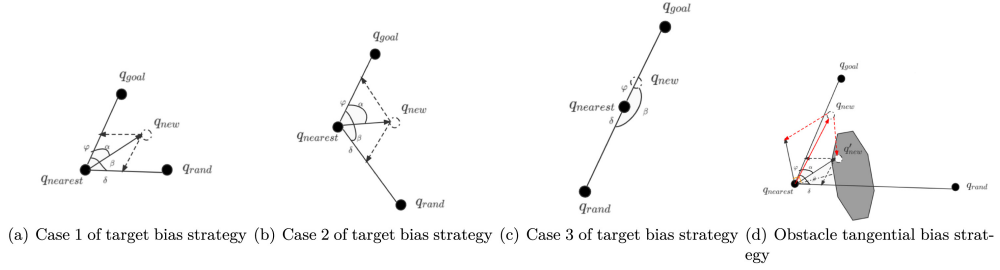


Figure 4.11: Illustration of the AFBG steering strategy. (a–c) Three cases of the goal-attraction bias, showing how the new node q_{new} is deflected toward q_{goal} depending on the relative positions of $q_{nearest}$, q_{rand} , and q_{goal} . (d) The obstacle-tangential bias redirects expansion tangentially along the obstacle boundary when $q_{nearest}$ is within 2δ of an obstacle.

When the tree node is far from obstacles, only the random direction and goal-attraction bias contribute, producing directed growth toward q_{goal} . When the tree node is close to an obstacle, the obstacle-tangential bias additionally deflects the expansion, transforming what would otherwise be an invalid expansion into a valid one that hugs the obstacle boundary. This latter property is particularly beneficial in narrow-passage environments, where paths must closely follow obstacle contours.

4.3.10 Integration into EBiRRT (SOF-EBiRRT)

In our EBiRRT framework, the standard STEER call is replaced by AFBG-STEER (Equation (4.18)) for both the forward tree $\mathcal{T}_S$ rooted at q_{start} and the

backward tree $\mathcal{T}_G$ rooted at q_{goal} . The AFBG steering is applied differently depending on which sampling strategy is active at each iteration.

When the guided sampling strategy is active, a sample q_{rand} is drawn from the restricted domain defined by the current guide node pair (s_{guide}, t_{guide}) . The tree is extended from the guide node $q_{choose} = q_{guide} \in \mathcal{T}$ rather than from a nearest-neighbor query to q_{rand} , focusing growth in the promising corridor between the two trees. For the start tree, $q_{choose} = s_{guide}$; for the goal tree, $q_{choose} = t_{guide}$. Within this strategy, the goal-bias term of the AFBG steer can be instantiated in two ways:

- **Cross-tree bias:** the goal-bias vector points from q_{guide} toward the corresponding *guide node in the opposite tree*, denoted q_{guide}^{opp} . Formally,

$$q_{goal}^{bias} = q_{guide}^{opp}, \quad (4.19)$$

where q_{guide}^{opp} is the node in $\mathcal{T}_G$ (resp. $\mathcal{T}_S$) paired with $q_{guide} \in \mathcal{T}_S$ (resp. $\mathcal{T}_G$) via the current guide path $\mathcal{P}_{guide}$. This variant directly attracts the extending node toward the meeting point of the two trees, maximizing the likelihood of a bidirectional connection within the guided corridor.

- **Goal-directed bias:** the goal-bias vector instead points toward the global goal configuration, i.e.,

$$q_{goal}^{bias} = \begin{cases} q_{goal}, & \text{for } \mathcal{T}_S, \\ q_{start}, & \text{for } \mathcal{T}_G. \end{cases} \quad (4.20)$$

This variant is preferred when the guide corridor is geometrically aligned with the global goal direction, as it simultaneously exploits both the local ellipsoidal constraint and the global goal attraction, reducing the risk of the guide path deviating into suboptimal regions.

In practice, the choice between cross-tree bias and goal-directed bias in Strategy

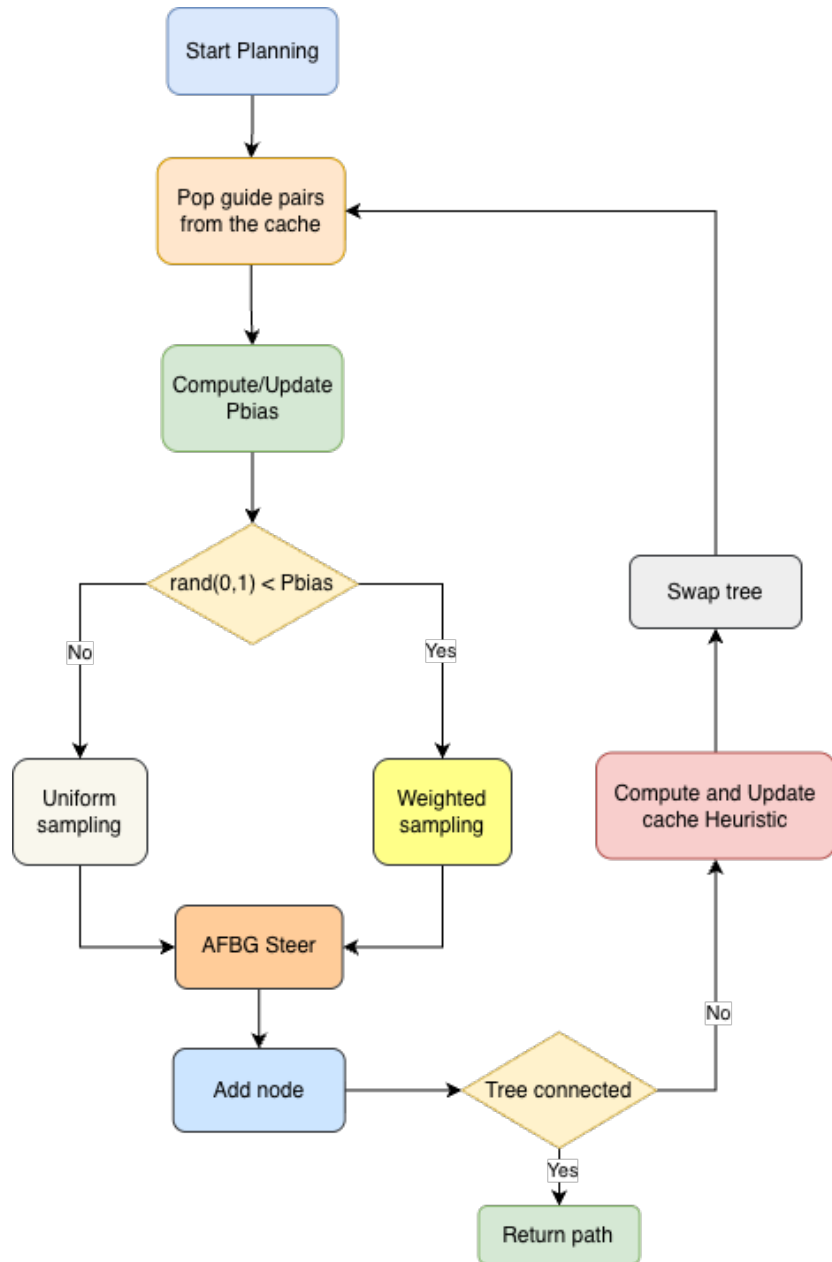


Figure 4.12: Workflow of the proposed SOF-EBiRRT planner, including guide-pair selection, adaptive bias computation, sampling strategy selection, AFBG steering, node insertion, tree connection checking, tree swapping, heuristic cache updating, and path return.

2 can be made adaptively: cross-tree bias is favored in early iterations when the two trees are far apart and a direct connection is the primary objective, while goal-directed bias becomes more beneficial in later iterations when the guide path is already close to optimal and fine-grained convergence is desired.

The combined effect of SOF sampling and AFBG steering across both strategies reduces the occurrence of invalid expansions, produces shorter and smoother initial solution paths, and accelerates overall convergence of the bidirectional planner, as demonstrated experimentally in Section 5.

Chapter 5

Experimental Results

This chapter presents the experimental evaluation of the proposed algorithms. We conducted comprehensive experiments in simulated 2D environments with varying obstacle densities to assess the performance of the baseline BiRRT algorithm alongside three enhanced variants: EBiRRT1, EBiRRT2, and SOF-EBiRRT. The evaluation includes environment setup details, visualization of planning results, and quantitative analysis of performance metrics.

5.1 Environment Setup

5.1.1 Environment setup

To evaluate the performance of the proposed algorithm, we conducted experiments in a simulated 2D environment. The algorithms were implemented in C++ and executed with Ubuntu 20.04. The specific hardware and software configurations are detailed in Table 5.1.

Table 5.1: Hardware and Software Configuration

Component	Specification
Operating System	Ubuntu 20.04 LTS
Programming Language	C++
CPU	Apple M2
RAM	16GB

5.1.2 Visualization Settings

To provide an intuitive assessment of the algorithms' performance, the simulation environment and planning results are visualized using the Robot Operating System (ROS) visualization package (RViz). The visualization parameters are configured to clearly distinguish between the static environment, the exploration trees, and the resultant paths of the different algorithms.

Map and Obstacle Configuration: The simulation environment is constructed by generating static obstacles within a bounded workspace. The map configuration is designed to mimic unstructured, cluttered scenarios with the following characteristics:

- **Obstacle Representation:** Obstacles are modeled as large, solid cubic structures. These structures are randomly distributed across the workspace based on a predefined density ratio. Unlike simple surface-based models, these obstacles are treated as volumetric entities, ensuring collision detection and preventing the planner from inadvertently traversing through internal voids.
- **Color:** In this work, the obstacles are rendered in **White**, while the free configuration space (C_{free}) is displayed with a **Dark Grey** background. This distinction aids in clearly identifying narrow passages and verifying the feasibility of the generated paths.

Path Visualization: To facilitate a comparative analysis of the three evaluated algorithms, we assign distinct colors to the final planned paths. This color-coding scheme allows for simultaneous visualization of multiple trajectories on the same map without ambiguity:

1. **Standard RRT Connect (Baseline):** Represented by a **White** path. This serves as the control group, typically exhibiting more zig-zag behavior and redundant exploration.
2. **EBi-RRT1 (Target-biased with Global Sampling):** Represented by a **Blue** path. This color indicates the trajectory resulting from the guide-pair heuristic combined with global random sampling.
3. **EBi-RRT2 (Target-biased with Circle Sampling):** Represented by a **Green** path. This color highlights the variant’s ability to focus the search strictly within the heuristic corridor, resulting in a more direct and efficient path than the baseline methods.
4. **SOF-EBiRRT (Integration of EBiRRT’s strategy and SOF, AFBG strategy):** Represented by a **Yellow** path. This highlights the ultimate proposed method, which leverages spatial obstacle forces to navigate dense clutter safely, consistently generating the shortest and most highly optimized route among all tested algorithms.

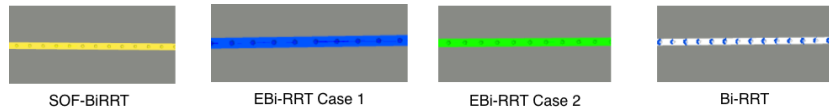


Figure 5.1: Path color legend used in the experimental visualizations: yellow for SOF-EBiRRT, blue for EBi-RRT Case 1, and green for EBi-RRT Case 2.

Figure 5.2 illustrates the visual difference between the paths generated by these three algorithms in a representative environment.

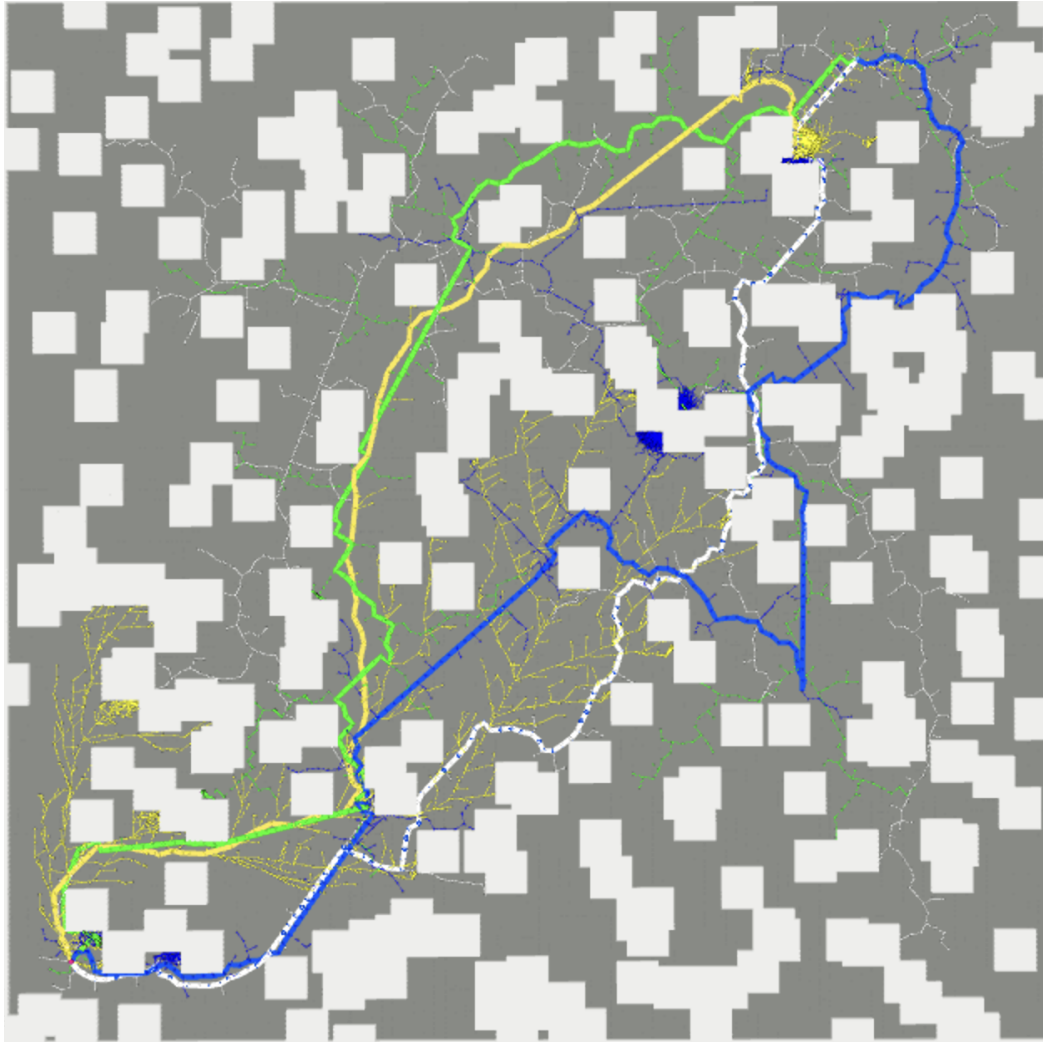


Figure 5.2: Visualization of the path planning results. The **White** blocks represent obstacles. The paths are colored as follows: **White** for Standard RRT Connect, **Blue** for EBi-RRT1, **Green** for EBi-RRT2, and **Yellow** for SOF-EBi-RRT

Figure 5.3 provides a closer look at the voxel-based obstacle generation used in the simulation code.

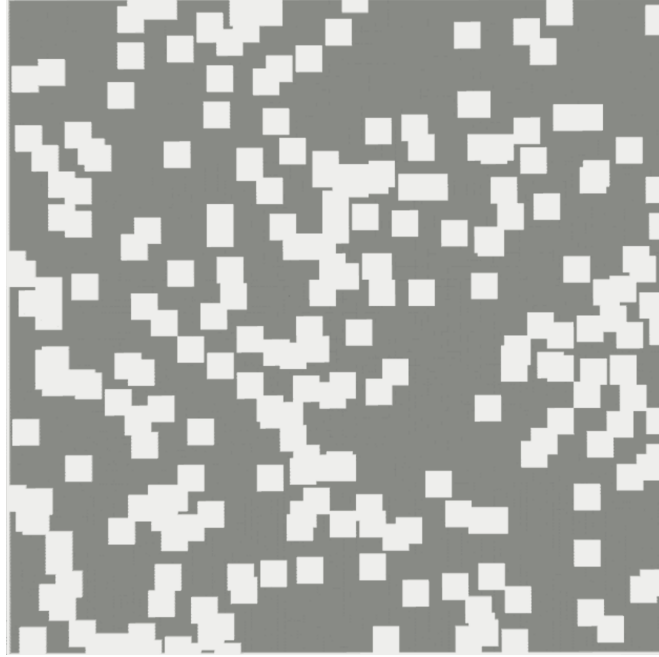


Figure 5.3: The generated map using random forest algorithm. Obstacles are constructed as squares distributed randomly across the map.

5.1.3 Parameter Settings

The parameters for the heuristic function and adaptive probability mechanism were configured to prioritize the connection between the two trees and encourage heuristic-guided expansion in the early stages. The specific values used in our experiments are summarized in Table 5.2.

Table 5.2: Algorithm Parameter Settings

Parameter	Symbol	Value
Heuristic Weight (Inter-tree)	α	7
Heuristic Weight (Start-to-Goal)	β	1
Heuristic Weight (Goal-to-Start)	γ	1
Initial Bias Probability	ρ_{init}	1
Tuning Factor	μ	2
Extension Step Size	ϵ	1.0
SOF-Weight grade	WG	1.0
N Blocks	N_{block}	8
Epsilon Floor	ϵ_{floor}	0.2
SOF Decay Rate	γ_{sof}	0.998
Lidar radius	r	15.0

5.1.4 Testing Scenarios

The testing environment was defined as a 500×500 space divided into 4×4 grid cells. We established three levels of complexity based on obstacle density:

- **Simple Environment:** Approximately 10% obstacle coverage.
- **Typical Environment:** Approximately 20% obstacle coverage.
- **Complex Environment:** Approximately 40% obstacle coverage.

For each difficulty level, 100 distinct pairs of start (q_{start}) and goal (q_{goal}) configurations were randomly generated to ensure statistical reliability.

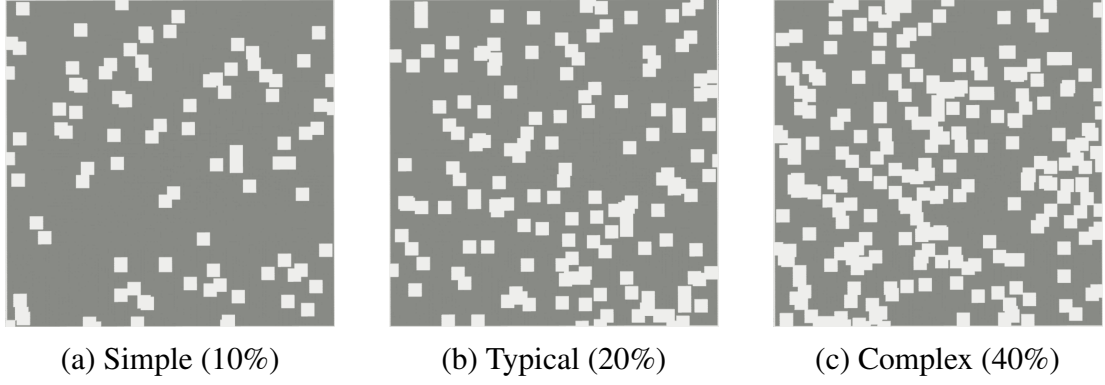


Figure 5.4: Visualization of the three testing environments with varying obstacle densities.

5.2 Result Visualization

5.2.1 Case 1: Simple Environment (10% Obstacles)

To better understand the behavior of the algorithms, we visualize the generated trees and paths in three representative scenarios. Figure 5.5 displays the planning results in a sparse environment.

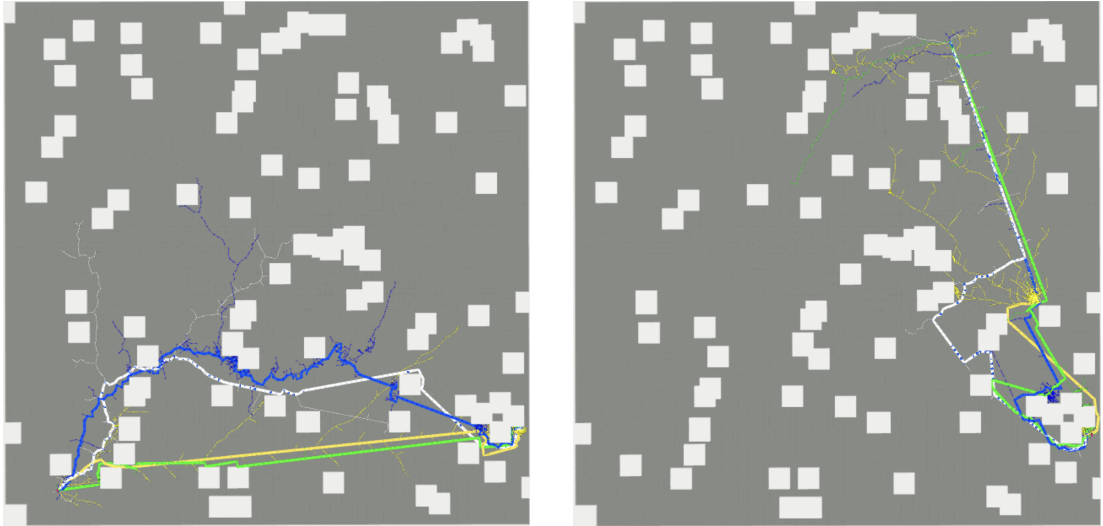


Figure 5.5: Visualization of path planning in the Simple Environment.

5.2.2 Case 2: Typical Environment (20% Obstacles)

Figure 5.6 illustrates the performance in an environment with moderate obstacle density.

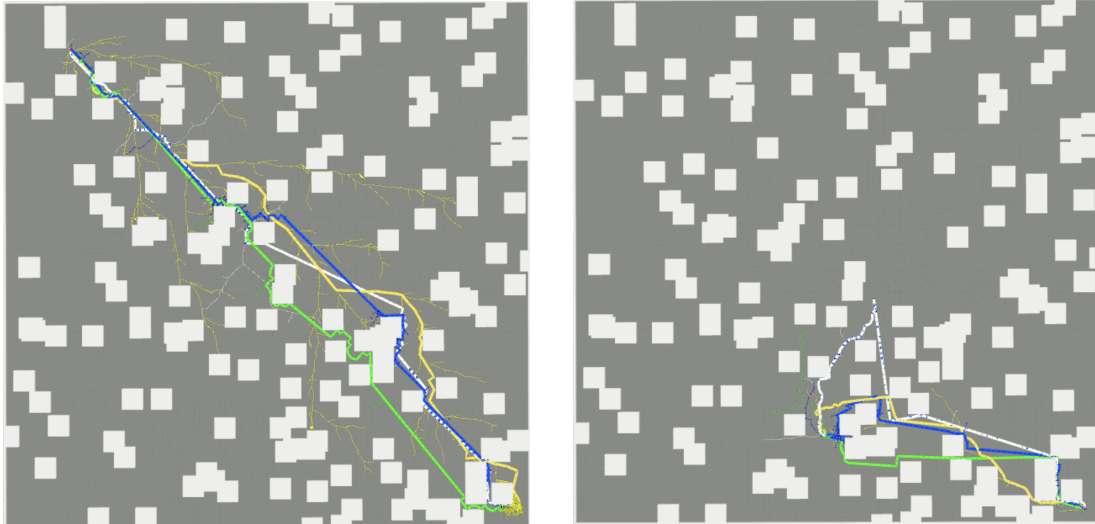


Figure 5.6: Visualization of path planning in the Typical Environment.

5.2.3 Case 3: Complex Environment (40% Obstacles)

Figure 5.7 demonstrates the algorithms' capability in a highly cluttered environment with narrow passages.

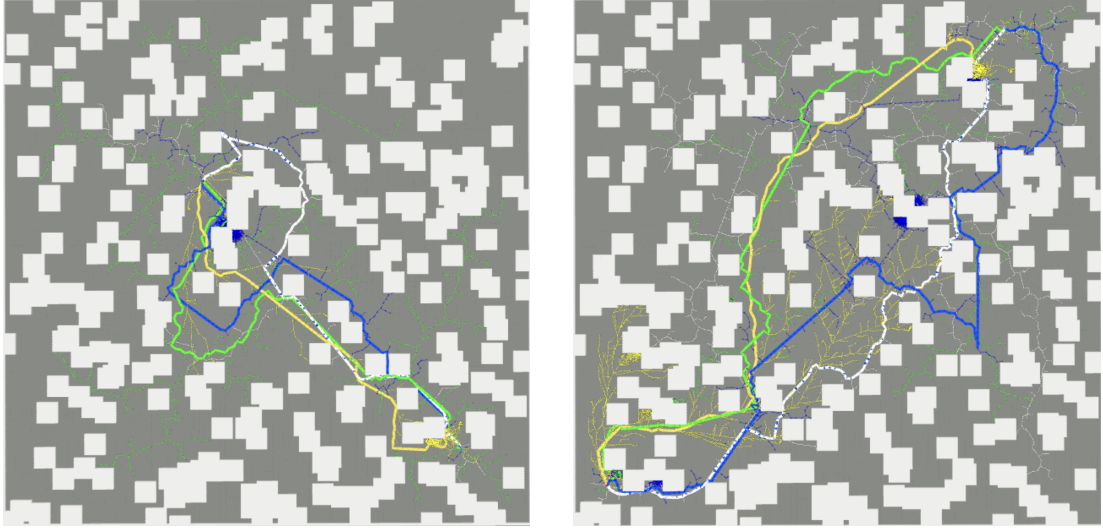


Figure 5.7: Visualization of path planning in the Complex Environment.

5.3 Result Analysis

5.3.1 Performance in Simple and Typical Environments

We evaluated the performance of the baseline BiRRT algorithm alongside three enhanced variants: EBi-RRT1, EBi-RRT2, and SOF-EBiRRT. The performance was assessed using five metrics: Success Rate (SR), Average Searching Time (AST), Average Path Length (APL), Average Number of Nodes (ANN), and Average Number of Iterations (ANI).

In environments with low to moderate obstacle density (Simple and Typical), all algorithms successfully navigated the space, achieving a **100% success rate**. The detailed comparisons are presented in Table 5.3 and Table 5.4.

- **Search Efficiency:** The baseline BiRRT demonstrated the lowest average searching time in both simple and typical environments. However, SOF-EBiRRT required fewer average nodes and iterations than the baseline in both cases. In simple environments, it reduced the average node count from 416.5200 to 381.6200 and the average iteration count from 318.0900 to 195.8900. In typical environments, it reduced the average node count from

458.63 to 353.98 and the average iteration count from 418.64 to 143.59.

- **Path Quality:** The enhanced variants, particularly SOF-EBiRRT, prioritized path optimization over sheer speed. SOF-EBiRRT consistently generated the shortest paths. For instance, in simple environments (Table 5.3), the average path length was reduced to 280.9958 compared to 292.5132 for BiRRT. This trend continued in typical environments, where SOF-EBiRRT achieved the shortest average path length of 277.1690, outperforming all other methods.

Table 5.3: Comparison of the BiRRT, EBi-RRT1, EBi-RRT2, SOF-EBiRRT in simple environments.

Performance metric	BiRRT	EBi-RRT1	EBi-RRT2	SOF-EBiRRT
Success rate	100%	100%	100%	100%
Average of searching time	0.0010	0.0717	0.0602	0.0106
Average path length	292.5132	287.4646	305.6271	280.9958
Average number of nodes	416.5200	6208.5859	7819.0707	381.6200
Average number of iterations	318.0900	791.5859	1290.4848	195.8900

Table 5.4: Comparison of the BiRRT, EBi-RRT1, EBi-RRT2, SOF-EBiRRT in typical environments.

Performance metric	BiRRT	EBi-RRT1	EBi-RRT2	SOF-EBiRRT
Success rate	100%	100%	100%	100%
Average of searching time	0.0012	0.0671	0.0544	0.0075
Average path length	293.4218	297.8175	297.7940	277.1690
Average number of nodes	458.63	8216.66	6958.59	353.98
Average number of iterations	418.64	1038.24	1061.84	143.59

5.3.2 Performance in Complex Environments

In the complex environment characterized by a 40% obstacle density, the algorithms showed different levels of robustness, as summarized in Table 5.5. SOF-EBiRRT achieved the highest success rate at **97.0%**, followed by the baseline BiRRT at 95.0%, while EBi-RRT1 and EBi-RRT2 achieved 77.0% and 60.0%, respectively.

The results highlight a clear trade-off between computational speed, robustness, and path quality. While the baseline BiRRT remained the fastest algorithm with an average searching time of **0.1043 s**, it produced the longest average path length of 381.0359.

In contrast, the SOF-EBiRRT algorithm proved its effectiveness in navigating heavy clutter and narrow passages by discovering the most optimal routes. It achieved the shortest average path length of **279.5952**, outperforming the baseline and both EBi-RRT variants. SOF-EBiRRT also required fewer iterations than the baseline BiRRT, reducing the average iteration count from 27,585.6842 to **11,022.2784**. This confirms that the sampling strategy of SOF-EBiRRT improves both path quality and robustness in highly dense environments.

Table 5.5: Comparison of the BiRRT, EBi-RRT1, EBi-RRT2, SOF-EBiRRT in complex environments (40% Obstacles).

Metric	BiRRT	EBi-RRT1	EBi-RRT2	SOF-EBiRRT
Success Rate (%)	95.0%	77.0%	60.0%	97.0%
Avg. Searching Time (s)	0.1043	52.5328	10.6656	0.4791
Avg. Path Length	381.0359	304.5019	301.2558	279.5952
Avg. Number of Nodes	2,434.1158	125,814.6753	85,010.8167	3,016.3299
Avg. Number of Iterations	27,585.6842	55,415.0390	43,783.0667	11,022.2784

Chapter 6

Conclusion

6.1 Summary

In this project, we addressed the limitations of traditional sampling-based path planning algorithms, which often prioritize fast exploration at the expense of path optimality in cluttered environments. We proposed an enhanced algorithm, named **SOF-EBiRRT**, which incorporates advanced steering and target-biased strategies to significantly improve path quality and safety.

The key contributions of this study are summarized as follows:

- **Heuristic Guidance and Adaptive Bias Mechanism:** We introduced a dynamic distance function and an adaptive bias to balance exploration and exploitation. While traditional methods may rush to a suboptimal solution, our approach guides the trees toward promising connection regions and helps the planner navigate complex obstacles more effectively.
- **SOF Sampling and AFBG Steering Strategy:** We enhanced the traditional uniform sampling and steering strategy with spatial probability-weight sampling and Artificial Potential Field Bias-Goal (AFBG) steering. This improves obstacle-aware exploration and goal-oriented expansion, directly contributing to shorter and safer paths in dense environments.

Overall, the results confirm a crucial trade-off in path planning: while baseline algorithms like BiRRT may converge faster, our proposed **SOF-EBiRRT** strategy invests additional computation to obtain shorter, safer, and higher-quality paths. This makes it suitable for applications where path quality and robustness are more important than raw planning speed.

6.2 Remarks

The experimental results show that each algorithm has a different strength. The baseline BiRRT is generally the fastest method because it performs simple random expansion and quickly accepts the first feasible connection. However, this speed often comes with longer and less optimized paths, especially in complex environments.

The two EBiRRT variants improve the search direction by using the heuristic guide pair (s_{guide}, t_{guide}) . EBiRRT1 provides more exploration because it expands from the guide node while still sampling globally. EBiRRT2 is more aggressive because it constrains sampling around the guide corridor, which can be effective when the corridor is valid but less robust when obstacles block the direct connection.

SOF-EBiRRT provides the best overall path quality in the tested environments. By using SOF sampling around the selected guide or exploratory node and applying AFBG steering, the planner can reduce invalid expansions near obstacles and produce shorter paths. The results also indicate a clear trade-off: SOF-EBiRRT improves robustness and path quality, but it may require more computation than the baseline because additional sampling weights, obstacle-distance checks, and steering biases must be evaluated.

6.3 Limitations

Although the proposed method improves path quality, several limitations remain:

- **Simulation-only evaluation:** The experiments were conducted in static 2D simulated environments. The method has not yet been validated on real mobile robots, with sensor noise, localization error, actuator constraints, or real-time system delays.
- **Computational overhead:** SOF sampling requires local ray casting and sector-weight computation, while AFBG steering requires obstacle-distance estimation. These additional steps increase the planning cost compared with the baseline BiRRT.
- **Parameter sensitivity:** The performance depends on parameters such as α , β , γ , ρ_{init} , WG , N_{block} , sensing radius, and step size. A parameter set that performs well in one obstacle density may not be optimal in another environment.
- **Limited environment diversity:** The evaluation focuses on randomly generated square obstacles with three density levels. More structured maps, dynamic obstacles, narrow maze-like passages, and non-convex obstacle layouts should be tested to assess generalization.
- **No formal optimality guarantee:** The proposed method improves empirical path length, but it does not yet include an RRT*-style rewiring mechanism or a formal asymptotic optimality proof.

6.4 Future Work

Future development of this project can focus on the following directions:

- **Computational optimization:** Improve the efficiency of SOF sampling and AFBG steering by caching obstacle-distance queries, using spatial indexing, reducing redundant ray casting, or parallelizing sector evaluation.
- **RRT*-inspired path optimization:** Integrate local parent selection and rewiring into SOF-EBiRRT so that path refinement occurs during planning rather than only after a feasible path is found.
- **Adaptive parameter tuning:** Design an automatic tuning mechanism for the guide-pair weights, sampling bias, SOF weight grade, and sensing radius based on obstacle density and recent expansion success rate.
- **Dynamic and real-world testing:** Extend the planner to handle moving obstacles and validate it on ROS-based robot platforms using realistic sensing, localization, and control constraints.
- **Extension to higher-dimensional planning:** Evaluate the approach in 3D workspaces and robot models with non-holonomic or kinodynamic constraints to better reflect real autonomous mobile robot navigation tasks.

References

- [1] J. Barraquand and J.-C. Latombe, “Robot motion planning: A distributed representation approach”, *The International Journal of Robotics Research*, vol. 10, no. 6, pp. 628–649, 1991.
- [2] S. M. LaValle, “Rapidly-exploring random trees: A new tool for path planning”, Computer Science Department, Iowa State University, Tech. Rep. TR 98-11, 1998.
- [3] L. E. Kavraki, P. Svestka, J.-C. Latombe, and M. H. Overmars, “Probabilistic roadmaps for path planning in high-dimensional configuration spaces”, *IEEE Transactions on Robotics and Automation*, vol. 12, no. 4, pp. 566–580, 1996.
- [4] S. M. LaValle and J. J. Kuffner, “Randomized kinodynamic planning”, in *Proceedings 1999 IEEE International Conference on Robotics and Automation (Cat. No.99CH36288C)*, vol. 1, 1999, pp. 473–479.
- [5] S. M. LaValle and J. J. Kuffner, “Randomized kinodynamic planning”, *The International Journal of Robotics Research*, vol. 20, no. 5, pp. 378–400, 2001.
- [6] J. J. Kuffner and S. M. LaValle, “RRT-connect: An efficient approach to single-query path planning”, in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2000, pp. 995–1001.

- [7] S. Karaman and E. Frazzoli, “RRT*: Optimal rapidly-exploring random tree algorithm”, in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, 2011, pp. 2561–2568.
- [8] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot, “Informed RRT*: Optimal sampling-based path planning focused via direct sampling of an admissible ellipsoidal heuristic”, in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Chicago, IL, USA, 2014, pp. 2997–3004.
- [9] G. Kang et al., “Sampling-based path planning with goal oriented sampling”, in *2016 IEEE International Conference on Advanced Intelligent Mechatronics (AIM)*, Banff, AB, Canada, 2016, pp. 1285–1290.
- [10] H. Liu, X. Zhang, J. Wen, R. Wang, and X. Chen, “Goal-biased bidirectional RRT based on curve-smoothing”, *IFAC-PapersOnLine*, vol. 52, no. 24, pp. 255–260, 2019, ISSN 2405-8963.
- [11] S. Lei, T. Li, X. Gao, et al., “Research on improved RRT path planning algorithm based on multi-strategy fusion”, *Scientific Reports*, vol. 15, p. 13 312, 2025.
- [12] S. Yu, J. Chen, G. Liu, X. Tong, and Y. Sun, “Sof-rrt*: An improved path planning algorithm using spatial offset sampling”, *Engineering Applications of Artificial Intelligence*, vol. 126, p. 106 875, 2023.